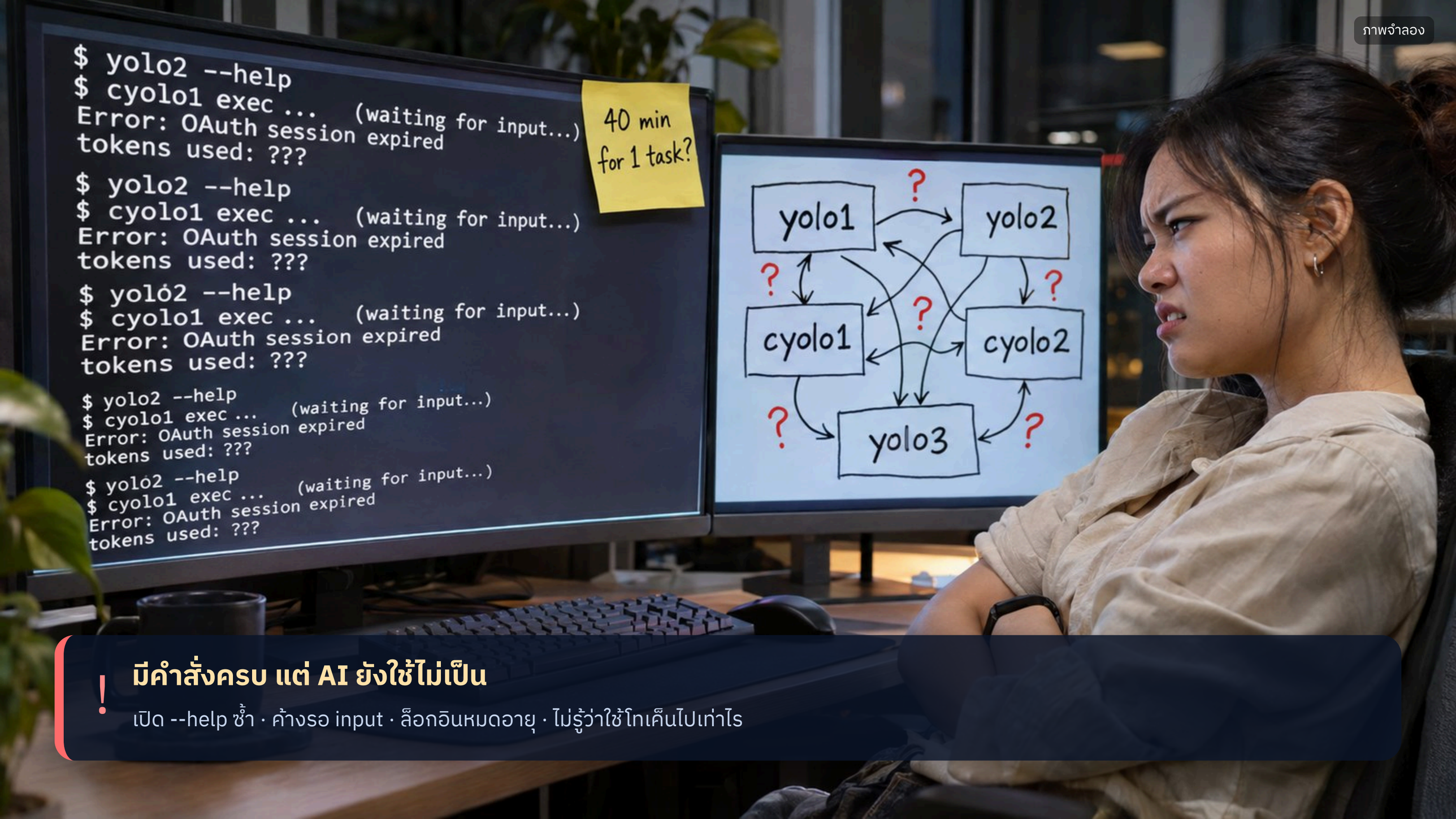
สัปดาห์ที่ 2 · 04_MultiAccount_Skill

การสร้างสกิล สำหรับระบบหลายบัญชี

yolo · cyolo · agent-team

ให้ AI ใช้ **yolo1–5** และ **cyolo1–5** ได้ถูกวิธี · ทำงานเป็นทีม · รู้สถิติทุกบัญชี · สลับหัวหน้าได้เมื่อโควตาหมด





มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น

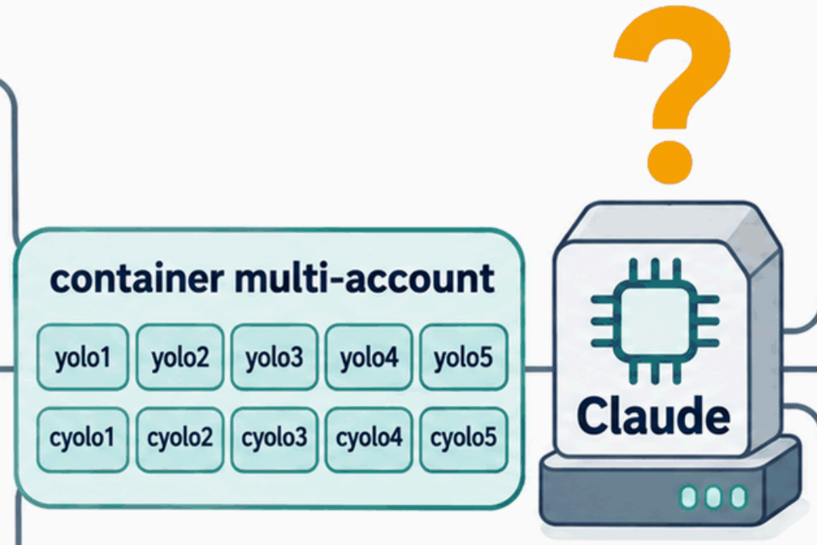
เปิด --help ซ้ำ · ค้างรอ input · ล็อกอินหมดอายุ · ไม่รู้ว่าใช้โทเค็นไปเท่าไร

มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น

เรียกผิดวิธี
ค้างรอ input · เปิดโหมดโต้ตอบ

สถานะล็อกอินไม่ตรงจริง
loggedIn = true แต่หมดอายุ

ประสานงานไม่เป็น
รอทีละตัว · ส่งข้อมูลยาวใน prompt



ไม่มีสถิติ
tokens · เวลา · โควตา ต้องเดา

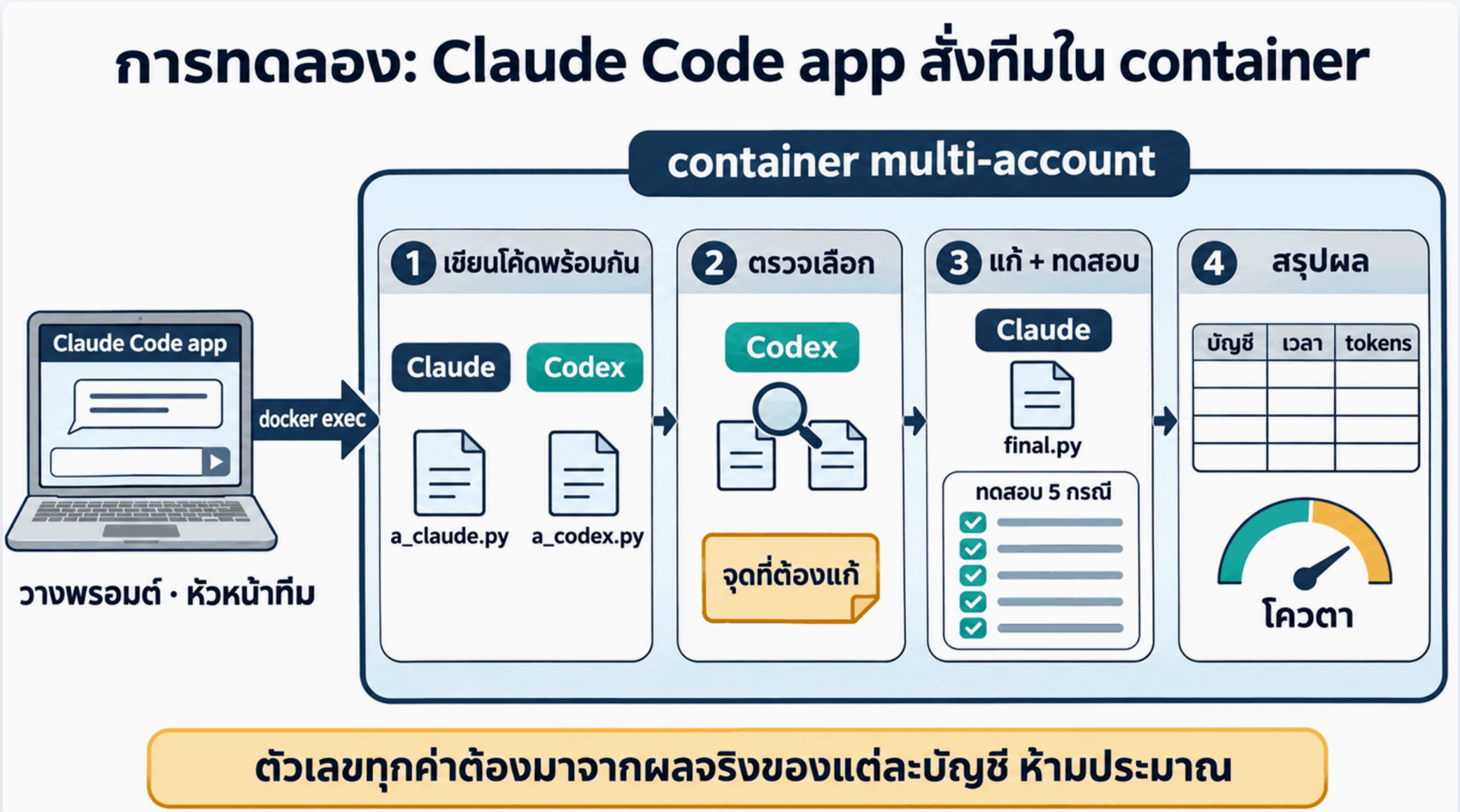
ลองผิดลองถูกทุกครั้ง
ช้า · เปลืองโทเค็น

หัวหน้าทีมโควตาหมด
แผนงานหาย · งานค้างกลางทาง

1 · ปัญหา

สิ่งที่เกิดขึ้นจริง

เพนพอยต์	สิ่งที่เกิดขึ้นจริง
เรียกผิดวิธี	`codex exec` ไม่ปิด stdin จะค้าง · ไม่ใส่ `-p` เปิดโหมดโต้ตอบ · exit code 0 ≠ สำเร็จ
สถานะล็อกอินไม่ตรงจริง	`auth status` ตอบ loggedIn แต่เรียกจริงได้ `OAuth session expired`
ประสานงานไม่เป็น	สั่งทีละบัญชี · ยัดข้อมูลยาวลง prompt · ไม่มีที่แชร์ความรู้
ไม่มีสถิติ	ไม่รู้โทเค็น เวลา และโควตาของแต่ละบัญชี
ลองผิดลองถูกทุกครั้ง	เปิด --help ซ้ำ เขียนสคริปต์ช่วยใหม่ทุกครั้ง
หัวหน้าทีมโควตาหมด	แผนงานอยู่ในบริบทแชตของหัวหน้า งานค้างหายไปด้วย


การทดลอง: Claude Code app สั่งทีมใน container
container multi-account
Claude Code app
docker exec
วางพรอมต์ · หัวหน้าทีม
1 เขียนโค้ดพร้อมกัน
Claude
Codex
a_claude.py
a_codex.py
2 ตรวจเลือก
Codex
จุดที่ต้องแก้
3 แก้ + ทดสอบ
Claude
final.py
ทดสอบ 5 กรณี
4 สรุปผล
บัญชี
เวลา
tokens
โควตา
ตัวเลขทุกค่าต้องมาจากผลจริงของแต่ละบัญชี ห้ามประมาณ

1.1 · พรอมต์ (ใช้ชุดเดียวกันทั้งก่อนและหลังมีสกิล)

```
ใช้ทีมเอเจนต์ใน container multi-account ทำงานต่อไปนี้ ทำงานใน container ผ่าน docker exec เท่านั้น (บัญชี yolo1-5 และ cyolo1-5 ใช้เฉพาะบัญชีที่พร้อม)
ก่อนเริ่ม ดูว่าใน container มีสกิลที่ช่วยงานนี้ได้หรือไม่ (/root/ai-accounts/.Claude/shared/skills) ถ้ามีให้อ่านและใช้
โฟลเดอร์งานใน container: /workspace/team-demo
1. ให้บัญชี Claude 1 บัญชี และบัญชี Codex 1 บัญชี เขียนฟังก์ชัน Python parse_duration(text) ที่แปลง "1h30m", "45s", "2h" เป็นจำนวนวินาที คนละแบบพร้อมกัน บันทึกเป็น a_claude.py และ a_codex.py
2. ส่งทั้งสองไฟล์ให้บัญชี Codex อีกบัญชีหนึ่งเลือกแบบที่ดีกว่าและบอกจุดที่ต้องแก้
3. ส่งคำแนะนำนั้นให้บัญชี Claude แก้เป็น final.py แล้วรันทดสอบ 5 กรณี
4. สรุปตาราง บัญชี | งาน | เวลา (วินาที) | input tokens | output tokens | สำเร็จ/ล้ม ตัวเลขต้องมาจากผลจริงของแต่ละบัญชี ห้ามประมาณ
5. บอกโควตาที่เหลือของแต่ละบัญชีที่ใช้
```

บรรทัด "ก่อนเริ่ม ดูว่าใน container มีสกิล..." ทำให้เทียบได้ยุติธรรม และเป็นวิธีที่ Claude Code app รู้จักสกิลใน container

1.1 · 💻 Claude Code app · ยังไม่มีสกิล



ภาพจำลองหน้าต่าง Claude Code app · ข้อความพร้อมต์และคำตอบตรงกับการรันจริง

1.1 · ผลจริงก่อนมีสกิล

810.2 วินาที · 30 คำสั่ง (ล้ม 9)

✻ **Claude** · 1.1 · ก่อนมีสกิล: Claude Code app สั่งทีมใน container · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

เสร็จครบทั้ง 5 ขั้น ทุกขั้นรันใน container `multi-account` ผ่าน `docker exec` ไฟล์งานอยู่ที่ `/workspace/team-demo`

สกิล: ใน `/root/ai-accounts/.Claude/shared/skills` มี devtools-lab, k8s-lab, reverse-engineer-anything, shared-check และ vastai แต่ไม่มีตัวไหนช่วยงานนี้ได้ จึงเขียนสคริปต์ `run_agent.py` เองไว้รันแต่ละบัญชีแบบไม่โต้ตอบ สคริปต์จับเวลาและอ่านจำนวน token จาก JSON ที่ CLI ส่งออกมา ผลดิบเก็บไว้ที่ `logs/*.json`

ผลแต่ละขั้น

1. **เขียนโค้ดพร้อมกัน:** yolo1 เขียน `a_claude.py` และ cyolo1 เขียน `a_codex.py` ทั้งสองใช้ regex `fullmatch` แบบเดียวกัน
2. **รีวิว:** cyolo2 เลือก **a_codex.py** เพราะใช้ `[0-9]` ซึ่งรับเฉพาะเลข ASCII ส่วน `\d` รับเลขยูนิโค้ดอื่นด้วย cyolo2 ตอบว่าไม่มีจุดที่ต้องแก้ตามสเปก
3. **final.py:** yolo2 ใช้ตรรกะเดิมของ a_codex.py ที่ปรับแค่ให้อ่านง่ายขึ้น (compile regex ไว้ครั้งเดียว และตั้งชื่อตัวคูณหน่วย) แล้วรัน `test_final.py` ได้ **5/5 passed** ผมรันซ้ำเองอีกรอบแล้วผลตรงกัน
4. กรณีทดสอบ: `1h30m` →5400, `45s` →45, `2h` →7200, `1h2m3s` →3723, `""` →ValueError

ตารางสรุป (ค่าจริงจาก log)

บัญชี	งาน	เวลา (วินาที)	input tokens	output tokens	สำเร็จ/ล้ม
yolo1 (Claude)	เขียน a_claude.py	29.08 (CLI วัดได้ 15.38)	92,978	584	สำเร็จ
cyolo1 (Codex)	เขียน a_codex.py	16.69	46,615	482	สำเร็จ
cyolo2 (Codex)	รีวิวและเลือกแบบ	12.16	36,309	226	สำเร็จ
yolo2 (Claude)	final.py และทดสอบ 5 กรณี	12.96 (CLI วัดได้ 10.34)	85,946	1,052	สำเร็จ (5/5)

วิธีอ่านตัวเลข

- **เวลา:** วัดตั้งแต่เริ่มจนจบโปรเซส ตัวเลขในวงเล็บคือ `duration_ms` ที่ Claude CLI รายงานเอง yolo1 ใช้เวลานานกว่าเพราะเริ่มพร้อม cyolo1 จึงรวมเวลาเปิดตัวด้วย
- **Input ของ Claude** = input + cache_creation + cache_read
- yolo1: 6 + 20,002 + 72,970
- yolo2: 6 + 17,674 + 68,266
- **Input ของ Codex** นับ cached รวมไว้แล้ว: cyolo1 cached 43,136 และ cyolo2 cached 30,080
- **ค่าใช้จ่ายที่ CLI รายงาน:** yolo1 $0.186, yolo2 $0.176

โควตาที่เหลือ (ดึงจริงหลังจบงาน เวลาเป็นเวลาไทย)

บัญชี	5 ชั่วโมง	7 วัน	หมายเหตุ
yolo1	ไม่มีลิมิตนี้	ใช้ 22% → **เหลือ 78%** (รีเซ็ต 12 พ.ย. 06:00)	—
yolo2	ใช้ 1% → **เหลือ 99%** (รีเซ็ต 8 พ.ย. 11:10)	ใช้ 2% → **เหลือ 98%** (รีเซ็ต 9 พ.ย. 13:00)	—
cyolo1	—	ใช้ 1% → **เหลือ 99%** (รีเซ็ต 14 พ.ย. ~12:21)	—
cyolo2	—	ใช้ 1% → **เหลือ 99%** (เหมือน cyolo1)	—

ไฟล์ทั้งหมดอยู่ใน `/workspace/team-demo`: `a_claude.py`, `a_codex.py`, `review.txt`, `final.py`, `test_final.py` และ `logs/`

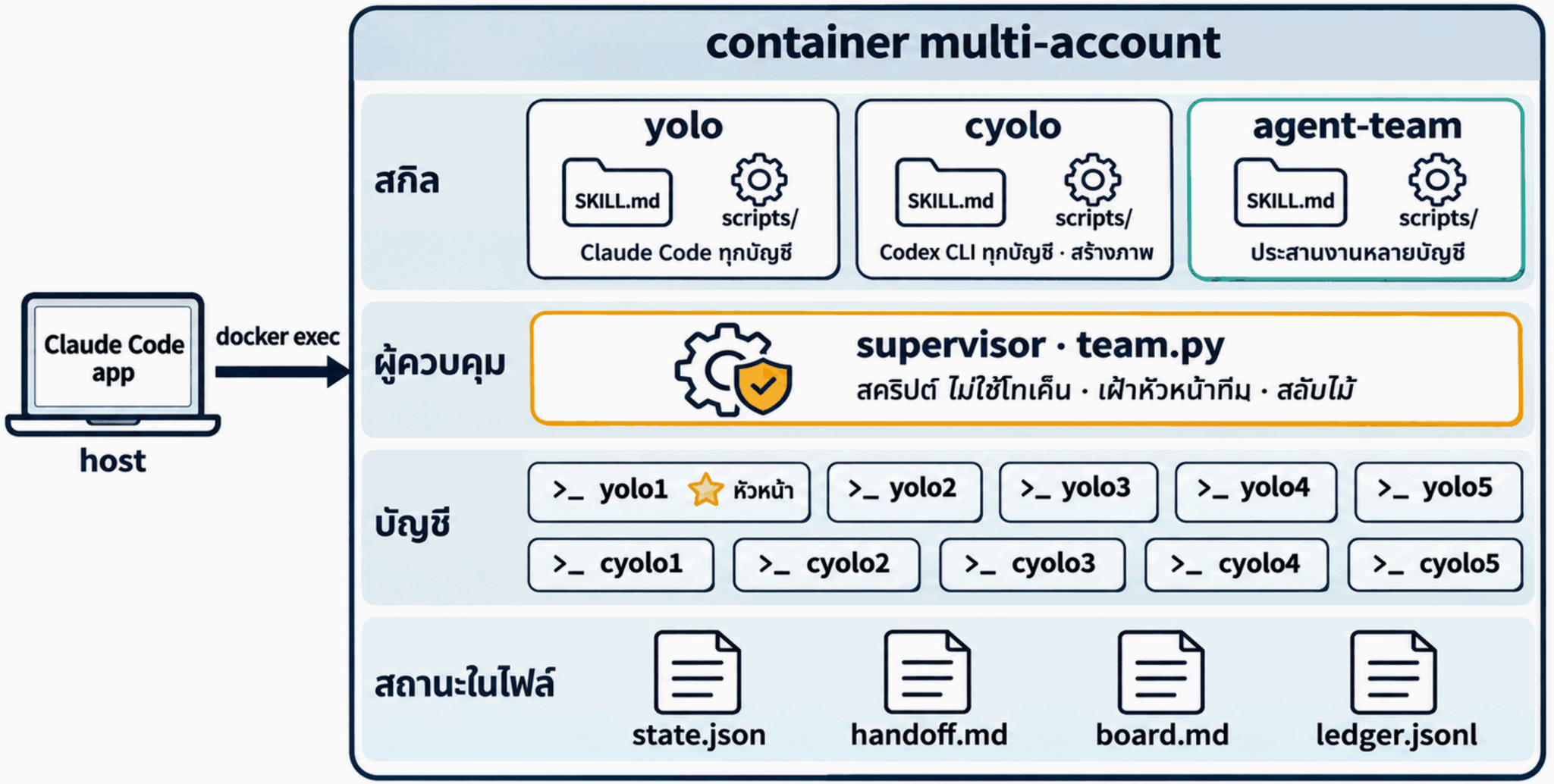
สิ่งที่สังเกตได้	รายละเอียด
สำรวจระบบเอง	ไล่เปิด container อื่น · Git Bash แปลง path จนล้ม
ยุ่งกับ credential	ตรวจ .credentials.json · **เปิด auth.json**
อ่านโค้ดเบื้องหลัง	อ่าน wrapper เพื่อเดาวิธีเรียก
เขียนเครื่องมือเอง	run_agent.py แล้ว docker cp เข้า container



งานซ้ำให้สคริปต์ · งานคิดให้ AI
เรียกบัญชีให้ถูก flag ตรวจความพร้อม จับเวลา รวมสถิติ = สคริปต์ในสกิล

เปิดทีละชั้น
AI เห็นแค่ชื่อและคำอธิบายตลอดเวลา เปิด SKILL.md เมื่อจะใช้ และสคริปต์เมื่อถึงขั้นนั้น

สถาปัตยกรรมของสกิล



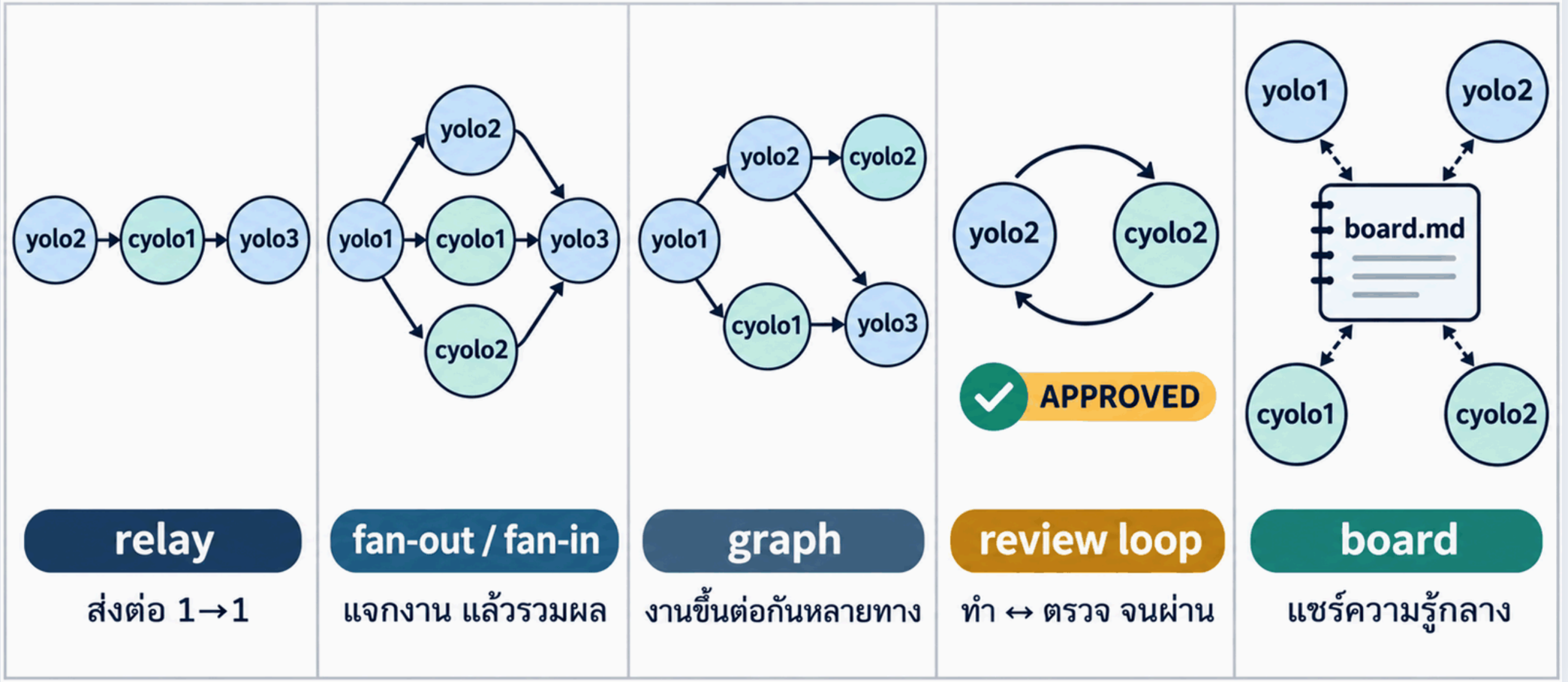
3 · ออกแบบสกิล 3 ตัว

แต่ละสกิลรู้เรื่องอะไร

สกิล	รู้เรื่องอะไร
`yolo`	Claude Code ทุกบัญชี · รุ่น/effort ตามความยาก · ตรวจความพร้อมด้วยการเรียกจริง · โควตา
`cyolo`	Codex ทุกบัญชี · รุ่นจาก models_cache · **สร้างภาพ** · โควตาจาก rollout
`agent-team`	ทำงานร่วมกันทั้งใน CLI เดียวกันและข้าม CLI · บริหารโทเค็น · สลับหัวหน้า · สถิติทั้งทีม

```
skills/
├── install.sh            ติดตั้งลง shared ของทั้งสอง CLI
├── README.md · TEST-REPORT.md
├── yolo/         SKILL.md · scripts/ · references/
├── cyolo/        SKILL.md · scripts/ · references/
└── agent-team/   SKILL.md · scripts/team.py · references/
```

รูปแบบการทำงานร่วมกันของหลายบัญชี



3.1 · รูปแบบการทำงานร่วมกัน



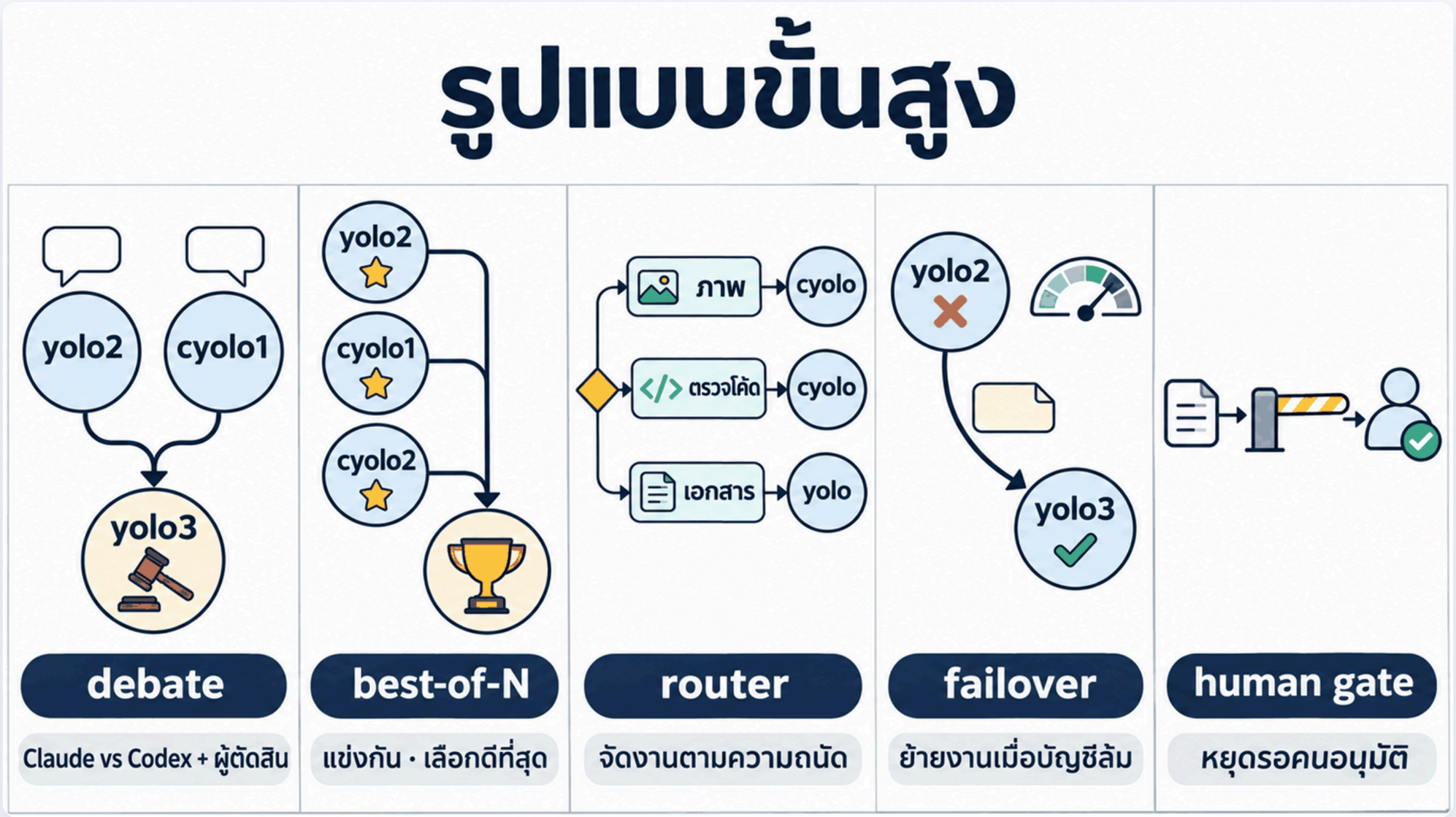
3.1 · รูปแบบการทำงานร่วมกัน

ใช้เมื่อไร และส่งข้อมูลอย่างไร

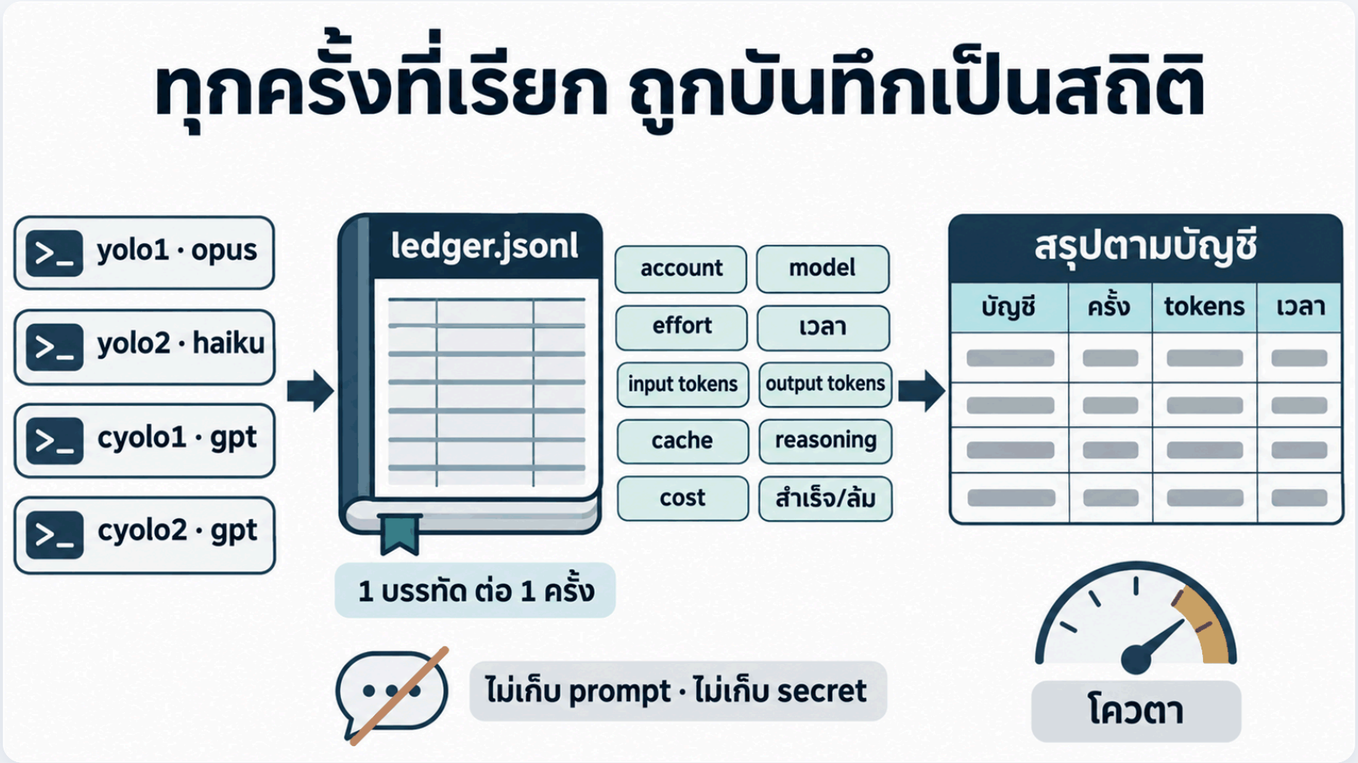
รูปแบบ	ใช้เมื่อ	การส่งข้อมูล
relay	งานเป็นลำดับ	`{{node}}` ค่าสั้นแทรกลง prompt
fan-out / fan-in	งานย่อยไม่ขึ้นต่อกัน	งานรวมผลอ่าน `{{file:node}}`
graph	งานขึ้นต่อกันหลายทาง	`depends_on` ตัวจัดคิวรันงานที่พร้อมให้เอง
review loop	ต้องการคุณภาพ	วนจน `APPROVED` หรือครบ max_rounds
board	แชร์ข้อค้นพบให้ agent ที่มาทีหลัง	อ่าน board.md ก่อนเริ่ม · เพิ่มต่อท้าย

> ข้อมูลยาวส่งเป็น **path ของไฟล์** ไม่ยัดลง prompt

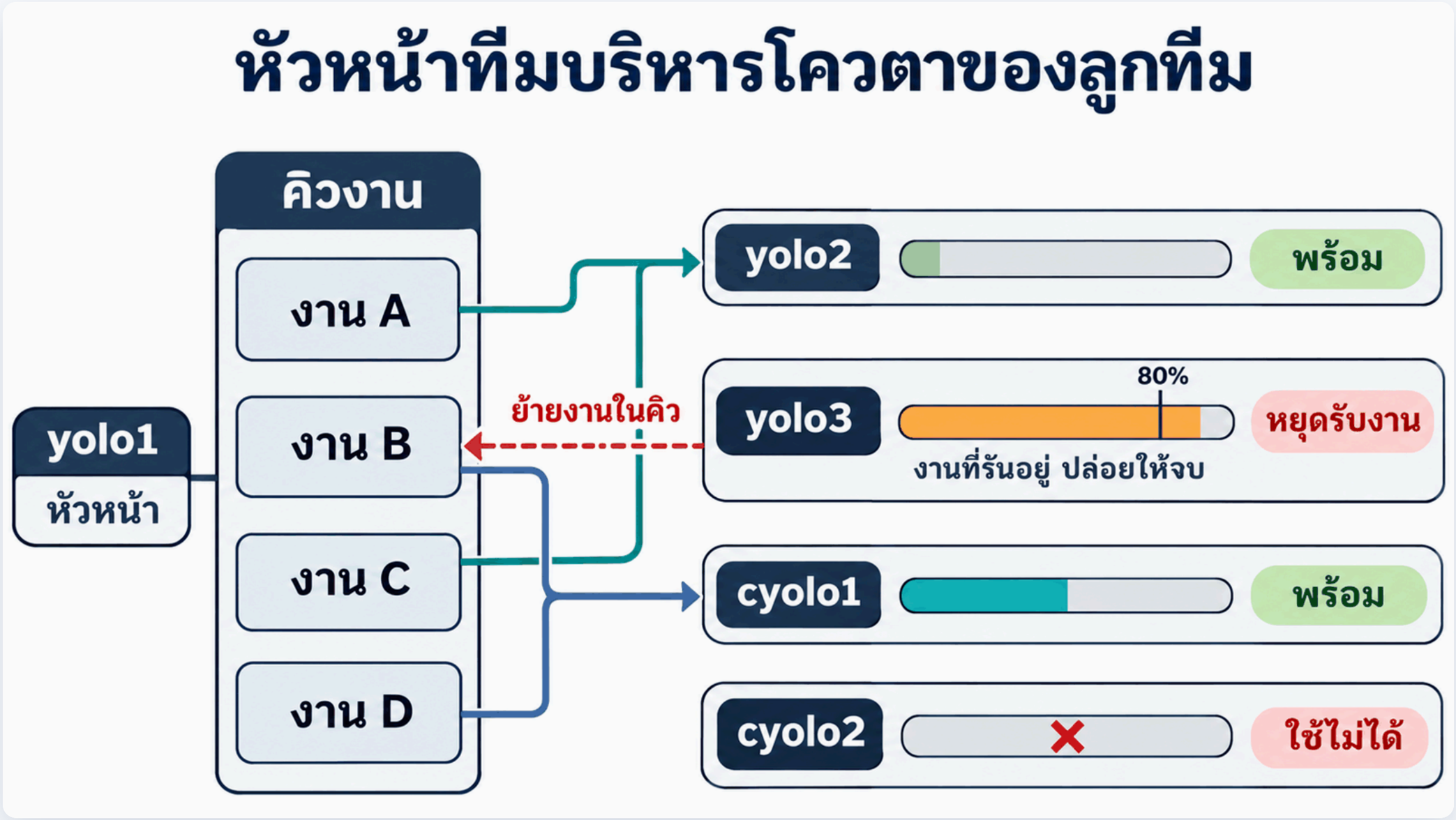
3.2 · รูปแบบขั้นสูง



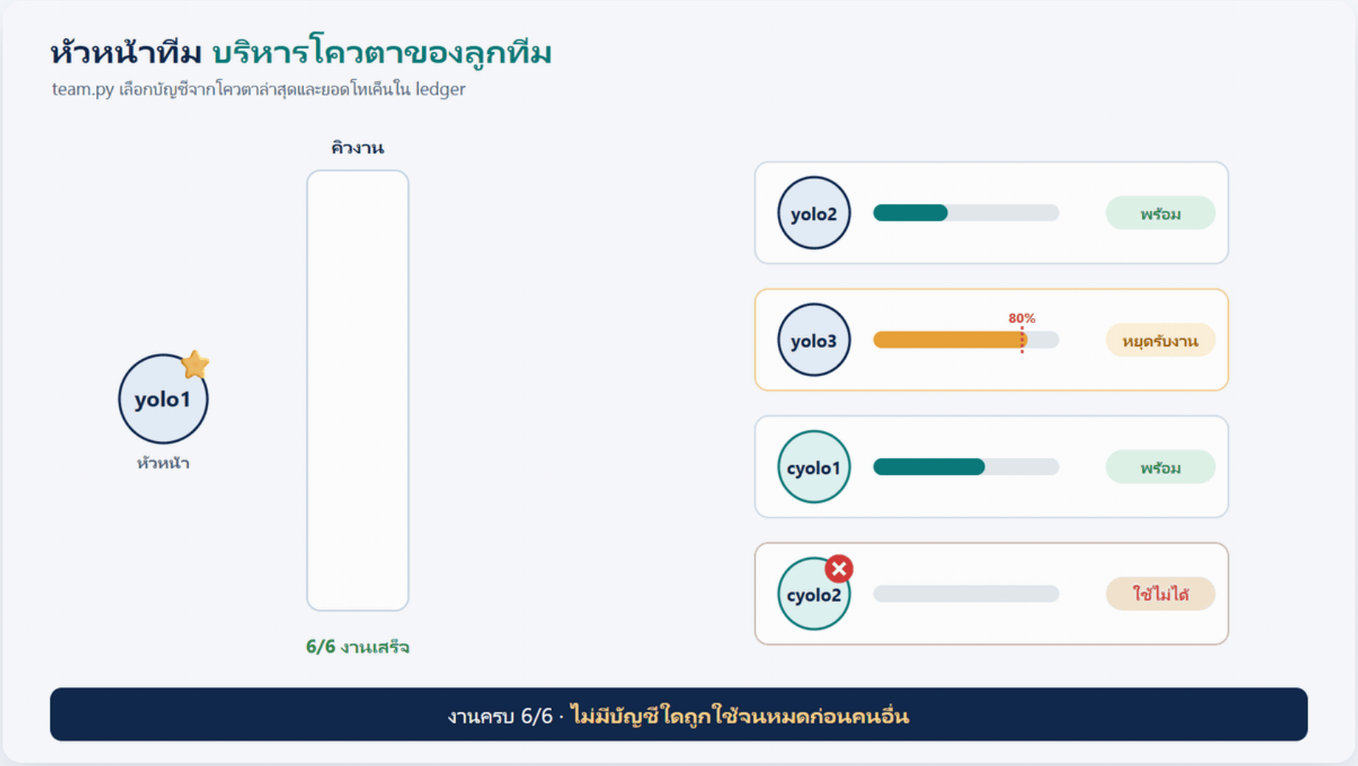
ต่อยอดได้: ทีมซ้อนทีม · โหวตเสียงข้างมาก · map-reduce · hedging · planner–executor–critic



- ทุกการเรียก = 1 บรรทัดใน `ledger.jsonl`
- `team.py stats` สรุปตามบัญชี / รุ่น / งาน
- `team.py quota` บอก**เวลาของข้อมูล**เสมอ
- **ไม่เก็บ prompt และ secret**


หัวหน้าทีมบริหารโควตาของลูกทีม
คิวงาน
งาน A
งาน B
งาน C
งาน D
yolo1
หัวหน้า
ย้ายงานในคิว
yolo2
พร้อม
yolo3
80%
หยุดรับงาน
งานที่รันอยู่ ปล่อยให้จบ
cyolo1
พร้อม
cyolo2
ใช้ไม่ได้

3.4 · หัวหน้าทีมบริหารโทเค็นของลูกทีม

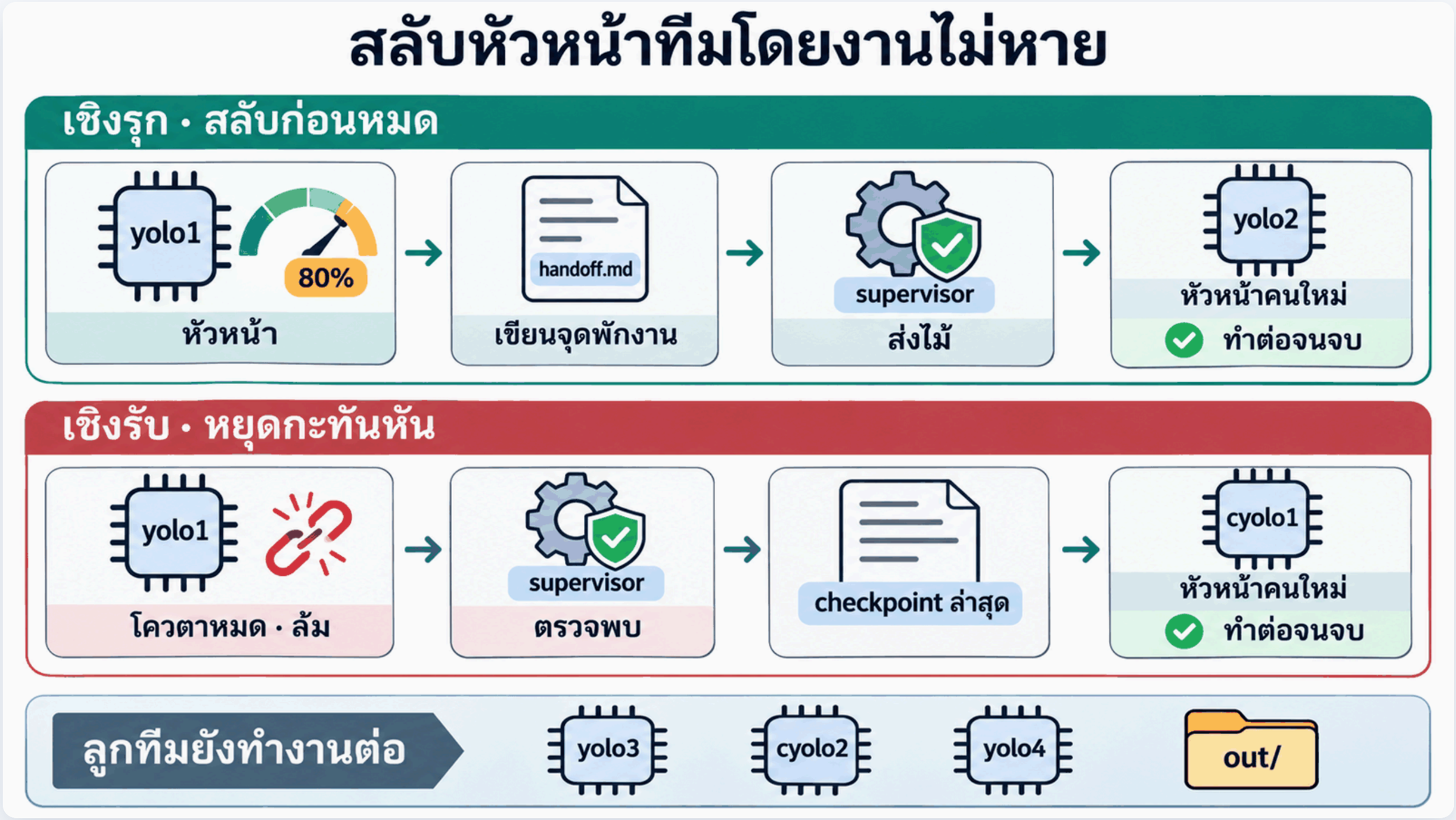


สถานการณ์	ตัวจัดคิวทำอะไร
ก่อนสั่งงาน	แจกให้บัญชีที่เหลือมากก่อน งานง่ายใช้รุ่นเล็ก
ใกล้หมด ≥ 80%	**หยุดรับงานใหม่** · งานในคิวย้าย
ล้มกลางคัน	ย้ายงาน + ความคืบหน้าใน msgs/
ผู้ใช้ระบุบัญชีเอง	ไม่ย้ายเงียบ ๆ หยุดและรายงาน

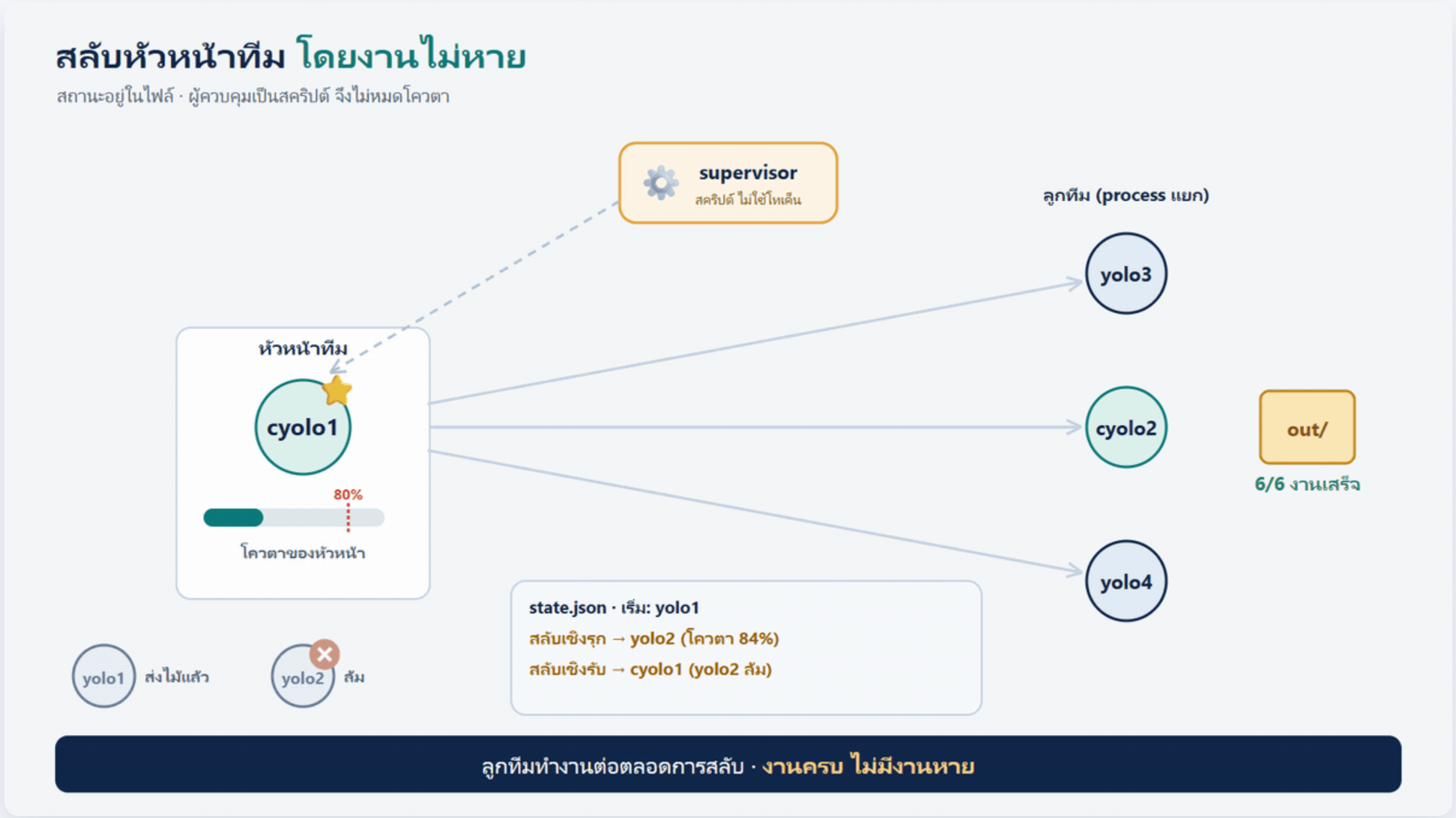


handoff.md = ใบส่งเวร

ก่อนออกเวรเขียนว่า ทำอะไรแล้ว · อะไรกำลังทำ · ต้องทำอะไรต่อ


สลับหัวหน้าทีมโดยงานไม่หาย
เชิงรุก · สลับก่อนหมด
yolo1
80%
หัวหน้า
handoff.md
เขียนจุดพักงาน
supervisor
ส่งไม้
yolo2
หัวหน้าคนใหม่
ทำต่อจนจบ
เชิงรับ · หยุดกะทันหัน
yolo1
โควตาหมด · ล้ม
supervisor
ตรวจพบ
checkpoint ล่าสุด
cyolo1
หัวหน้าคนใหม่
ทำต่อจนจบ
ลูกทีมยังทำงานต่อ
yolo3
cyolo2
yolo4
out/

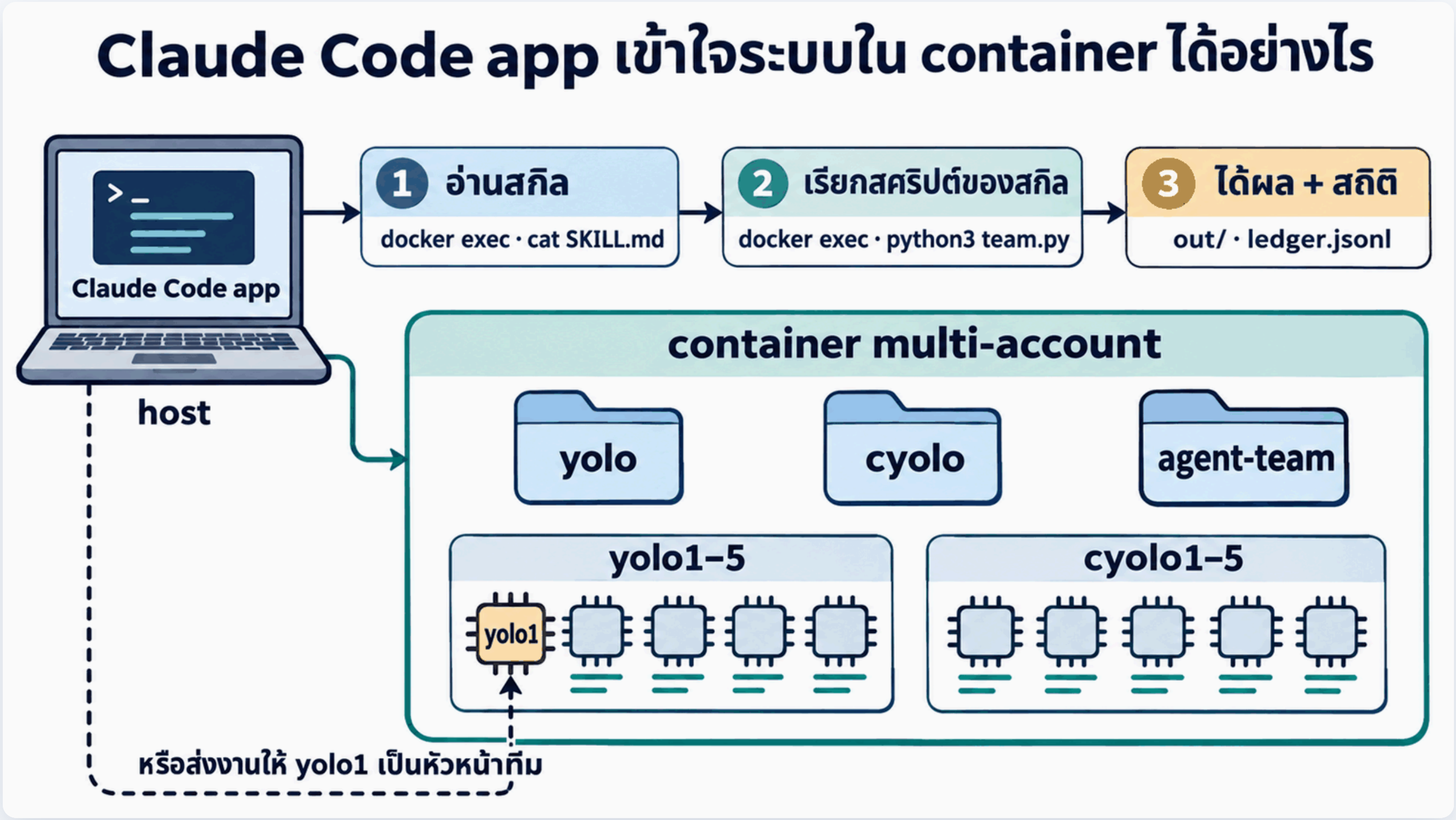
3.5 · สลับหัวหน้าทีมโดยงานไม่หาย



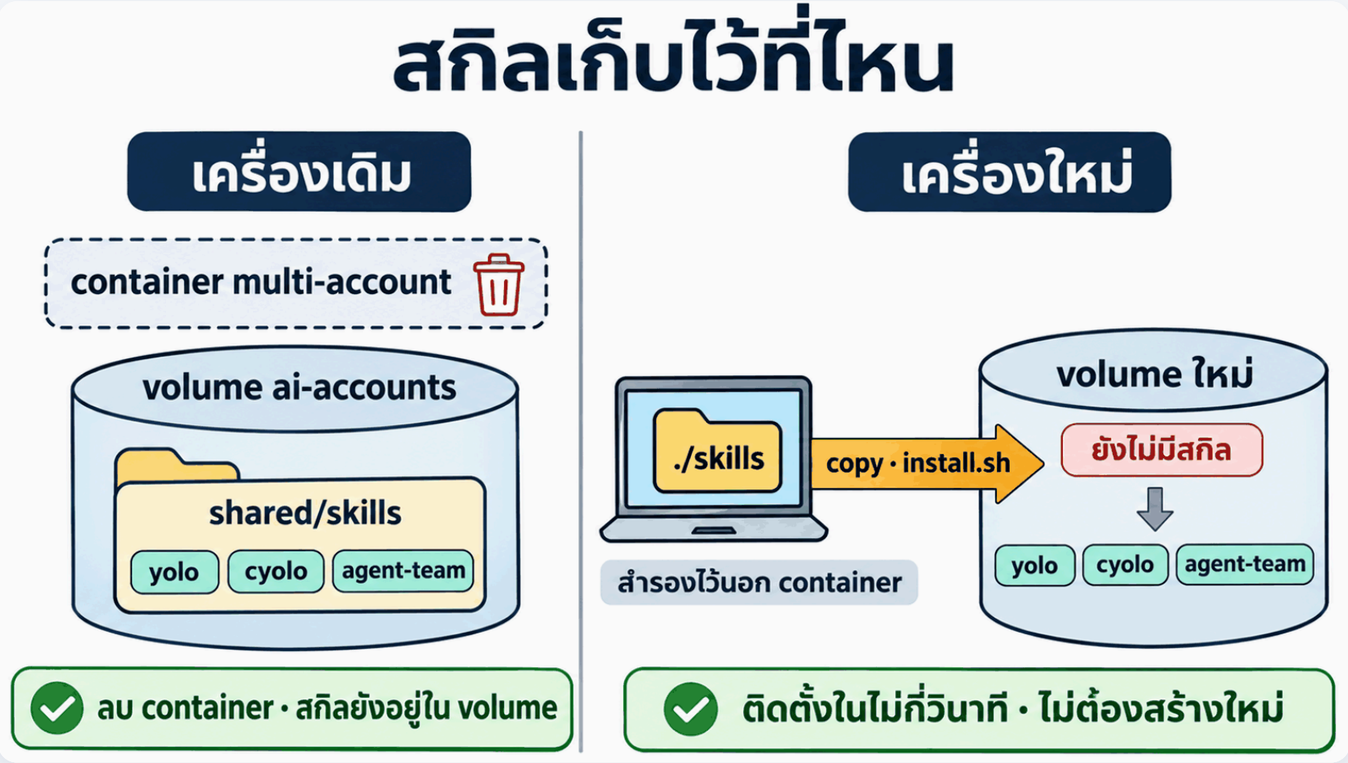
3.5 · สลับหัวหน้าทีมโดยงานไม่หาย

หลักการออกแบบ

หลักการ	รายละเอียด
สถานะอยู่ในไฟล์	state.json · handoff.md
ผู้ควบคุมเป็นสคริปต์	`team.py lead` ไม่ใช้โทเค็น · หัวหน้าทำงานเป็นรอบ เขียน checkpoint ทุกรอบ
ลูกทีมไม่ผูกกับหัวหน้า	process แยก · หัวหน้าเปลี่ยนตัว งานไม่หาย
สลับเชิงรุก	โควตา ≥ 80% หรือเกินงบ → ส่งไม้ตั้งแต่ยังเขียน checkpoint ทัน
สลับเชิงรับ	ล้ม · rate limit · เงียบไป → เริ่มคนใหม่จาก checkpoint
ผู้สืบทอด	พร้อม + โควตาเหลือมาก · ฝั่งเดียวกันก่อน (หัวหน้าเป็น Codex ได้)

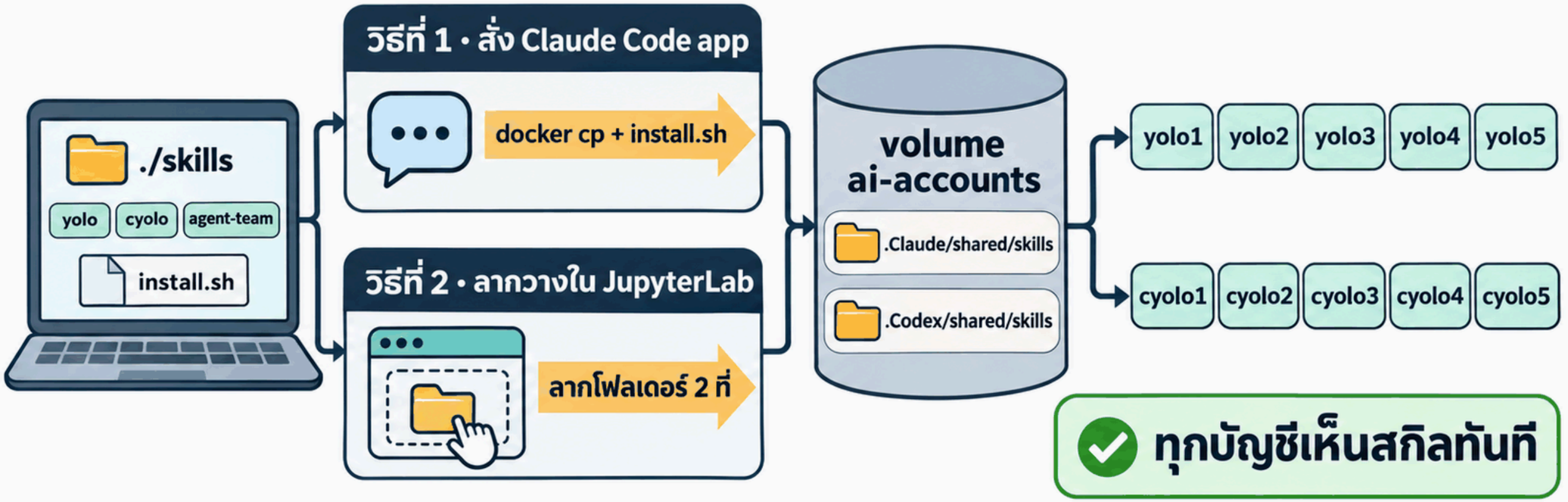

Claude Code app เข้าใจระบบใน container ได้อย่างไร
Claude Code app
host
1 อ่านสกิล
docker exec · cat SKILL.md
2 เรียกสคริปต์ของสกิล
docker exec · python3 team.py
3 ได้ผล + สถิติ
out/ · ledger.jsonl
container multi-account
yolo
cyolo
agent-team
yolo1–5
yolo1
cyolo1–5
หรือส่งงานให้ yolo1 เป็นหัวหน้าทีม

4 · สกิลเก็บไว้ที่ไหน



ที่เก็บ	ลบ container	เครื่องใหม่
โฟลเดอร์สำรอง `skills/`	ยังอยู่	**นำไปติดตั้งได้**
`shared/skills` ใน volume	ยังอยู่	ไม่ติดไป
container	หายไป	ไม่ติดไป

ติดตั้งสกิลจากโฟลเดอร์สำรอง 2 วิธี



5 · ภาพรวมขั้นตอนการทดลอง

ขั้น	ทำที่ไหน	เวลารันจริง
1 สร้างสกิลจากสเปก	Claude Code app	~33 นาที
2 ติดตั้งจากโฟลเดอร์สำรอง	Claude Code app	39 วินาที
3 งานทีมชุดเดิมอีกครั้ง	Claude Code app	181.9 วินาที
4 สลับหัวหน้า + บริหารโทเค็น	Claude Code app	~3 นาที
5 ติดตั้งบนเครื่องใหม่	Claude Code app	1.59 วินาที

```
04_MultiAccount_Skill/
├── prompt_skill_spec.md   ← สเปกของสกิล 3 ตัว (input ที่ 1)
├── reference/             ← สกิล yolo / cyolo ชุดเดิม ใช้เป็นแหล่งความรู้ (input ที่ 2)
└── _solution/skills/      ← สกิลที่ได้จากการรันจริง = โฟลเดอร์สำรองพร้อมติดตั้ง
```

ต้องมี container multi-account และล็อกอินอย่างน้อย 1 บัญชี Claude + 1 บัญชี Codex

1

ให้ Claude สร้างสกิลจากสเปก

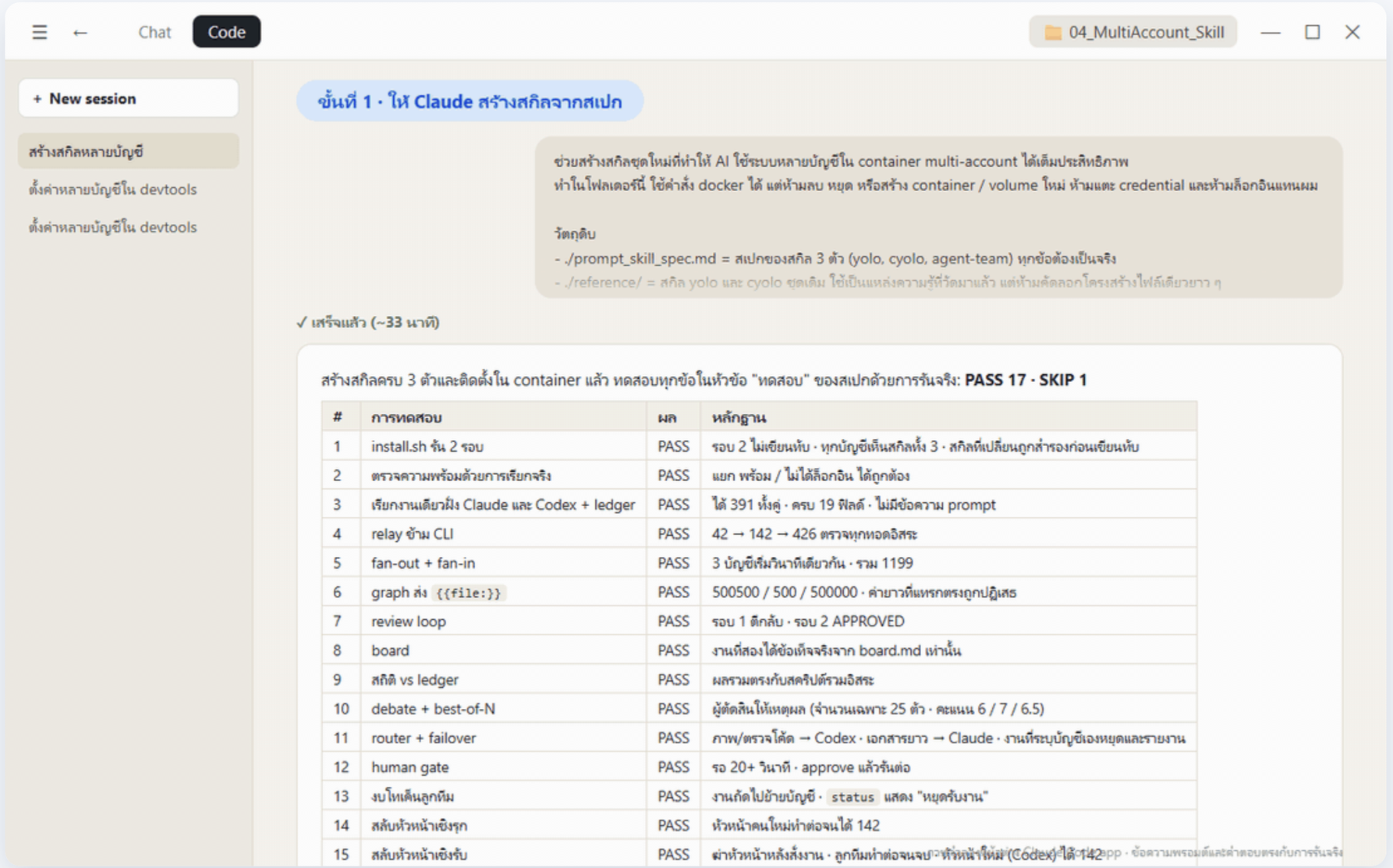
input: **prompt_skill_spec.md** + **reference/** (สกิลเดิม) → **./skills** (~33 นาที)

ขั้นที่ 1 · input

หัวข้อในสเปก	สาระ
เป้าหมาย	3 สกิล · SKILL.md + scripts/ + references/ · Python มาตรฐาน
ผู้ใช้ 2 ฝั่ง	ใน container และ Claude Code app บน host (docker exec)
ข้อเท็จจริงจากการทดลอง	flag ที่ถูก · ปิด stdin · exit code ไม่ใช่ตัวบอกผล · auth status ไม่ตรงจริง · ตำแหน่งโควตา
agent-team	run directory · พื้นฐาน 5 แบบ · ขั้นสูง 5 แบบ · บริหารโทเค็น · สลับหัวหน้า
ledger / install.sh / ทดสอบ	ฟิลด์ที่เก็บ · ติดตั้งซ้ำได้ · เกณฑ์ PASS/FAIL/SKIP

reference/ = สกิล yolo / cyolo ชุดเดิม (ไฟล์เดียวยาว ~44 KB ต่อสกิล) ใช้เป็นแหล่งความรู้

ขั้นที่ 1 · 💻 Claude Code app



ขั้นที่ 1 · พรอมต์ (คัดลอกทั้งกรอบ)

```
ช่วยสร้างสกิลชุดใหม่ที่ทำให้ AI ใช้ระบบหลายบัญชีใน container multi-account ได้เต็มประสิทธิภาพ
ทำในโฟลเดอร์นี้ ใช้คำสั่ง docker ได้ แต่ห้ามลบ หยุด หรือสร้าง container / volume ใหม่ ห้ามแตะ credential และห้ามล็อกอินแทนผม

วัตถุดิบ
- ./prompt_skill_spec.md = สเปกของสกิล 3 ตัว (yolo, cyolo, agent-team) ทุกข้อต้องเป็นจริง
- ./reference/ = สกิล yolo และ cyolo ชุดเดิม ใช้เป็นแหล่งความรู้ที่วัดมาแล้ว แต่ห้ามคัดลอกโครงสร้างไฟล์เดียวยาว ๆ
- container multi-account ที่กำลังรัน = ที่ทดสอบจริง (บัญชีที่พร้อมอาจมีไม่ครบ ให้ตรวจเอง)

สร้างไว้ใน ./skills (นอก container นี่คือต้นฉบับที่เก็บสำรอง)
ทดสอบ: คัดลอก ./skills ไปที่ /tmp/skills ใน container แล้วรัน install.sh ติดตั้งลง shared ทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปก
ใช้บัญชีอย่างประหยัด (haiku/low กับงานทดสอบ) ข้อที่ต้องใช้บัญชีที่ไม่พร้อม = SKIP
เขียน skills/TEST-REPORT.md แล้วสรุปเป็นตารางสั้น ๆ: ผลทดสอบ ไฟล์ที่สร้าง และเวลาที่ใช้ แล้วหยุด
```

ขั้นที่ 1

หลักการออกแบบพรอมต์

ให้ข้อเท็จจริงที่ตรวจแล้ว

สเปกรวมกับดักจากการรันจริง Claude ไม่ต้องลองผิดลองถูกเอง

ห้ามคัดลอกโครงสร้างเดิม

สกิลใหม่ต้องแยกชั้นตามหลักการเปิดทีละชั้น

ต้นฉบับอยู่นอก container

./skills คือโฟลเดอร์สำรองที่นำไปติดตั้งเครื่องใหม่ได้

ทดสอบแบบประหยัด

งานทดสอบใช้รุ่นเล็ก และจำลองโควตาหมดแทนการใช้จริง

ขั้นที่ 1 · ผลจริง 8 พ.ย. 2026

PASS 17 · SKIP 1

✻ **Claude** · ขั้นที่ 1 · ให้ Claude สร้างสกิลจากสเปก · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

สร้างสกิลครบ 3 ตัวและติดตั้งใน container แล้ว ทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปกด้วยการรันจริง: **PASS 17 · SKIP 1**

#	การทดสอบ	ผล	หลักฐาน
1	install.sh รัน 2 รอบ	PASS	รอบ 2 ไม่เขียนทับ · ทุกบัญชีเห็นสกิลทั้ง 3 · สกิลที่เปลี่ยนถูกสำรองก่อนเขียนทับ
2	ตรวจความพร้อมด้วยการเรียกจริง	PASS	แยก พร้อม / ไม่ได้ล็อกอิน ได้ถูกต้อง
3	เรียกงานเดียวฝั่ง Claude และ Codex + ledger	PASS	ได้ 391 ทั้งคู่ · ครบ 19 ฟิลด์ · ไม่มีข้อความ prompt
4	relay ข้าม CLI	PASS	42 → 142 → 426 ตรวจทุกทอดอิสระ
5	fan-out + fan-in	PASS	3 บัญชีเริ่มวินาทีเดียวกัน · รวม 1199
6	graph ส่ง `{{file:}}`	PASS	500500 / 500 / 500000 · ค่ายาวที่แทรกตรงถูกปฏิเสธ
7	review loop	PASS	รอบ 1 ตีกลับ · รอบ 2 APPROVED
8	board	PASS	งานที่สองได้ข้อเท็จจริงจาก board.md เท่านั้น
9	สถิติ vs ledger	PASS	ผลรวมตรงกับสคริปต์รวมอิสระ
10	debate + best-of-N	PASS	ผู้ตัดสินให้เหตุผล (จำนวนเฉพาะ 25 ตัว · คะแนน 6 / 7 / 6.5)
11	router + failover	PASS	ภาพ/ตรวจโค้ด → Codex · เอกสารยาว → Claude · งานที่ระบุบัญชีเองหยุดและรายงาน
12	human gate	PASS	รอ 20+ วินาที · approve แล้วรันต่อ
13	งบโทเคินลูกทีม	PASS	งานถัดไปย้ายบัญชี · `status` แสดง "หยุดรับงาน"
14	สลับหัวหน้าเชิงรุก	PASS	หัวหน้าคนใหม่ทำต่อจนได้ 142
15	สลับหัวหน้าเชิงรับ	PASS	ฆ่าหัวหน้าหลังสั่งงาน · ลูกทีมทำต่อจนจบ · หัวหน้าใหม่ (Codex) ได้ 142
16	เรียกจาก host ผ่าน `docker exec`	PASS	yolo.py, team.py (graph ทาง stdin), cyolo.py image
17	สร้างภาพผ่าน cyolo	PASS	PNG จริง · เป็นแมวสีส้มตามสั่ง
18	ข้อที่ต้องใช้บัญชีที่ไม่พร้อม	SKIP	ไม่มีบัญชีที่อยู่ในสถานะ "ล็อกอินแล้วแต่หมดอายุ" ให้ทดสอบ

ข้อสังเกตจากการทดสอบ

- haiku/low เคยตอบ 12+30 ผิดหนึ่งครั้ง จึงต้องตรวจผลอิสระตามสเปก
- Codex 0.167.0 ที่ไม่ได้ล็อกอินออกด้วย exit 1 (สเปกบอก 0) สคริปต์ไม่พึ่ง exit code จึงไม่กระทบ
- Git Bash บน Windows แปลง path `/root/...` ที่ส่งให้ `docker exec` เพิ่มหมายเหตุใน SKILL.md ทุกตัวแล้ว

ไฟล์ที่สร้าง (`./skills`): `install.sh` · `README.md` · `TEST-REPORT.md` · `yolo/` · `cyolo/` · `agent-team/`

ใช้งานรวม 103 ครั้ง (10 ครั้งที่ล้มเป็นการจำลองโดยตั้งใจ) · เวลา 06:38–07:10 ประมาณ 32 นาที

คำตอบจริงเป็นภาษาอังกฤษ แปลและจัดหน้าใหม่ · SKIP = ไม่มีบัญชีในสถานะ "ล็อกอินแล้วแต่หมดอายุ" ให้ทดสอบ

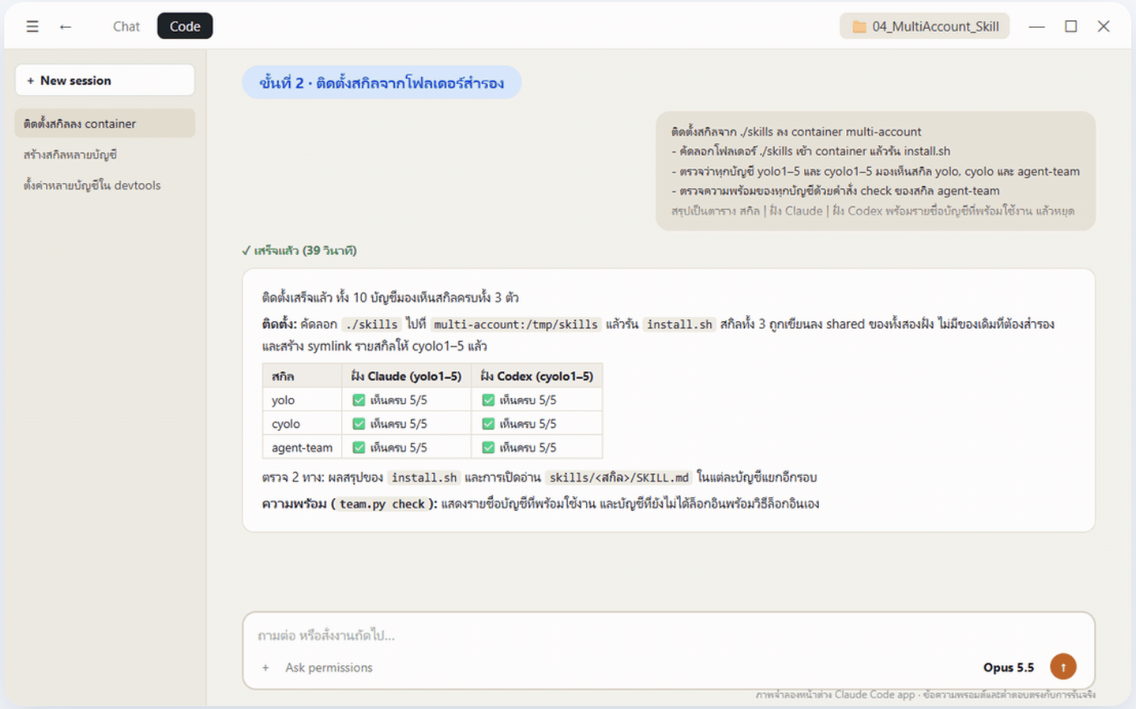
2

ติดตั้งสกิลจากโฟลเดอร์สำรอง

docker cp + **install.sh** หรือลากวางใน JupyterLab (39 วินาที)

ขั้นที่ 2 · วิธีที่ 1 · 💻 Claude Code app

```
ติดตั้งสกิลจาก ./skills ลง container multi-account
- คัดลอกโฟลเดอร์ ./skills เข้า container แล้วรัน install.sh
- ตรวจว่าทุกบัญชี yolo1-5 และ cyolo1-5 มองเห็นสกิล yolo, cyolo และ agent-team
- ตรวจความพร้อมของทุกบัญชีด้วยคำสั่ง check ของสกิล agent-team
สรุปเป็นตาราง สกิล | ฝั่ง Claude | ฝั่ง Codex พร้อมรายชื่อบัญชีที่พร้อมใช้งาน แล้วหยุด
```



ขั้นที่ 2 · ผลจริง

✻ **Claude** · ขั้นที่ 2 · ติดตั้งสกิลจากโฟลเดอร์สำรอง · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ติดตั้งเสร็จแล้ว ทั้ง 10 บัญชีมองเห็นสกิลครบทั้ง 3 ตัว

ติดตั้ง: คัดลอก `./skills` ไปที่ `multi-account:/tmp/skills` แล้วรัน `install.sh` สกิลทั้ง 3 ถูกเขียนลง shared ของทั้งสองฝั่ง ไม่มีของเดิมที่ต้องสำรอง และสร้าง symlink รายสกิลให้ cyolo1–5 แล้ว

สกิล	ฝั่ง Claude (yolo1–5)	ฝั่ง Codex (cyolo1–5)
yolo	✅ เห็นครบ 5/5	✅ เห็นครบ 5/5
cyolo	✅ เห็นครบ 5/5	✅ เห็นครบ 5/5
agent-team	✅ เห็นครบ 5/5	✅ เห็นครบ 5/5

ตรวจ 2 ทาง: ผลสรุปของ `install.sh` และการเปิดอ่าน `skills/<สกิล>/SKILL.md` ในแต่ละบัญชีแยกอีกรอบ

ความพร้อม (`team.py check`): แสดงรายชื่อบัญชีที่พร้อมใช้งาน และบัญชีที่ยังไม่ได้ล็อกอินพร้อมวิธีล็อกอินเอง

วิธีที่ 2 · ลากวางใน JupyterLab

ลาก yolo, cyolo, agent-team **ทีละโฟลเดอร์** ไปวาง 2 ที่

`.Claude/shared/skills`	yolo1–5
`.Codex/shared/skills`	cyolo1–5

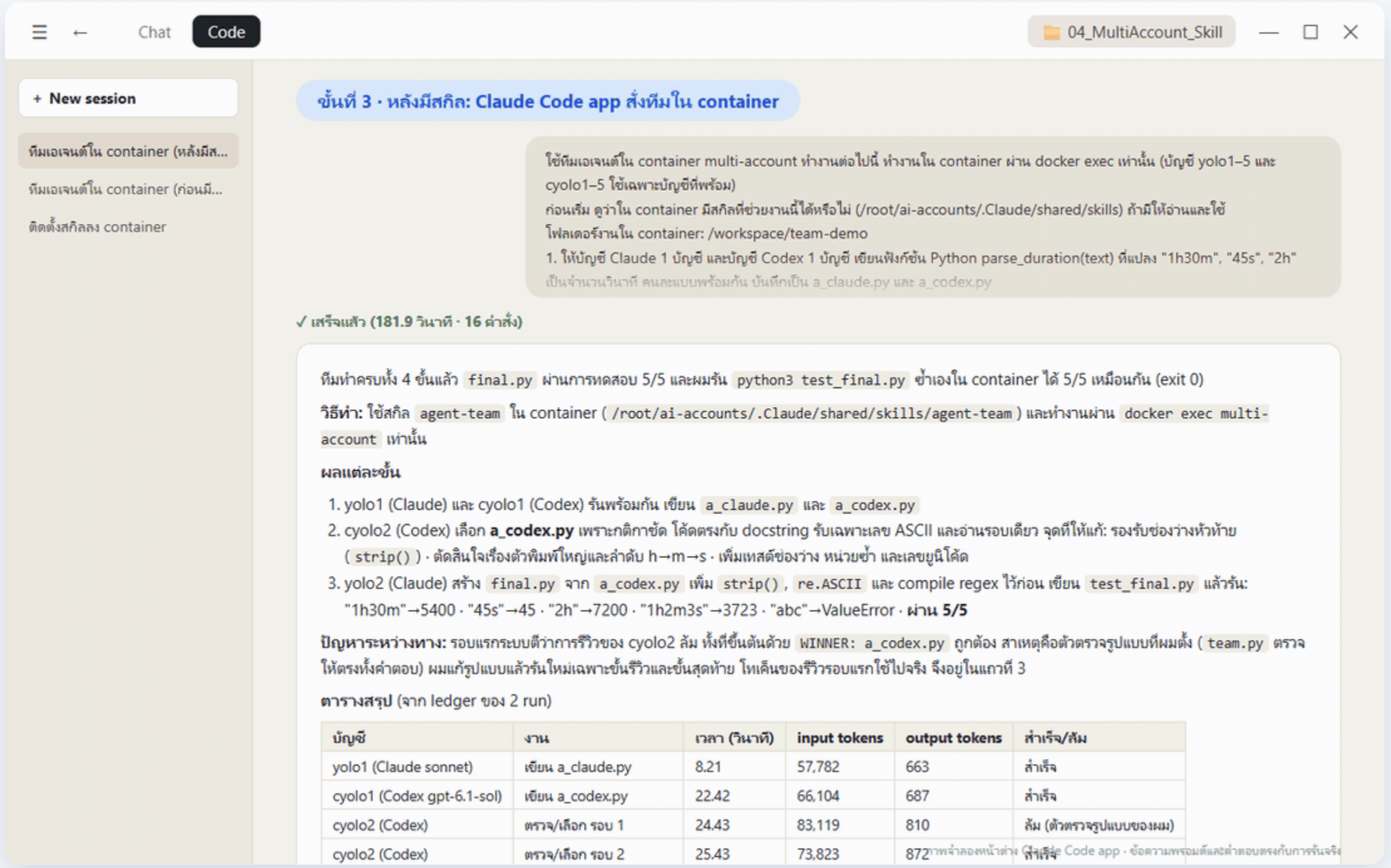
ใช้สกิลที่เตรียมไว้: เปลี่ยน ./skills เป็น ./_solution/skills

3

ทดสอบงานทีมชุดเดิมอีกครั้ง

วางพรอมต์ชุดเดียวกับหัวข้อ 1.1 ใน **Claude Code app** · ตอนนี้ใน container มีสกิลแล้ว (181.9 วินาที)

ขั้นที่ 3 · 💻 Claude Code app · มีสกิลแล้ว



ขั้นที่ 3 · ผลจริงหลังมีสกิล

✻ **Claude** · ขั้นที่ 3 · หลังมีสกิล: Claude Code app สั่งทีมใน container · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ทีมทำครบทั้ง 4 ขั้นแล้ว `final.py` ผ่านการทดสอบ 5/5 และผมรัน `python3 test_final.py` ซ้ำเองใน container ได้ 5/5 เหมือนกัน (exit 0)

วิธีทำ: ใช้สกิล `agent-team` ใน container (`/root/ai-accounts/.Claude/shared/skills/agent-team`) และทำงานผ่าน `docker exec multi-account` เท่านั้น

ผลแต่ละขั้น

1. yolo1 (Claude) และ cyolo1 (Codex) รันพร้อมกัน เขียน `a_claude.py` และ `a_codex.py`
2. cyolo2 (Codex) เลือก **a_codex.py** เพราะกติกาชัด โค้ดตรงกับ docstring รับเฉพาะเลข ASCII และอ่านรอบเดียว จุดที่ให้แก้: รองรับช่องว่างหัวท้าย (`strip()`) · ตัดสินใจเรื่องตัวพิมพ์ใหญ่และลำดับ h→m→s · เพิ่มเทสต์ช่องว่าง หน่วยซ้ำ และเลขยูนิโค้ด
3. yolo2 (Claude) สร้าง `final.py` จาก `a_codex.py` เพิ่ม `strip()` , `re.ASCII` และ compile regex ไว้ก่อน เขียน `test_final.py` แล้วรัน: "1h30m"→5400 · "45s"→45 · "2h"→7200 · "1h2m3s"→3723 · "abc"→ValueError · **ผ่าน 5/5**

ปัญหาระหว่างทาง: รอบแรกระบบตีว่าการรีวิวของ cyolo2 ล้ม ทั้งที่ขึ้นต้นด้วย `WINNER: a_codex.py` ถูกต้อง สาเหตุคือตัวตรวจรูปแบบที่ผมตั้ง (`team.py` ตรวจให้ตรงทั้งคำตอบ) ผมแก้รูปแบบแล้วรันใหม่เฉพาะขั้นรีวิวและขั้นสุดท้าย โทเคนของรีวิวรอบแรกใช้ไปจริง จึงอยู่ในแถวที่ 3

ตารางสรุป (จาก ledger ของ 2 run)

บัญชี	งาน	เวลา (วินาที)	input tokens	output tokens	สำเร็จ/ล้ม
yolo1 (Claude sonnet)	เขียน a_claude.py	8.21	57,782	663	สำเร็จ
cyolo1 (Codex gpt-6.1-sol)	เขียน a_codex.py	22.42	66,104	687	สำเร็จ
cyolo2 (Codex)	ตรวจ/เลือก รอบ 1	24.43	83,119	810	ล้ม (ตัวตรวจรูปแบบของผม)
cyolo2 (Codex)	ตรวจ/เลือก รอบ 2	25.43	73,823	872	สำเร็จ
yolo2 (Claude sonnet)	final.py + ทดสอบ 5 กรณี	14.21	86,221	1,571	สำเร็จ (5/5)

โควตาที่เหลือ (`team.py quota --fresh`)

บัญชี	ใช้ไป	คงเหลือโดยประมาณ
yolo1	สัปดาห์ (ทุกรุ่น) 22%	~78%
yolo2	session 1% · สัปดาห์ 2%	~98%
cyolo1	7 วัน 1%	~99%
cyolo2	7 วัน 1%	~99%

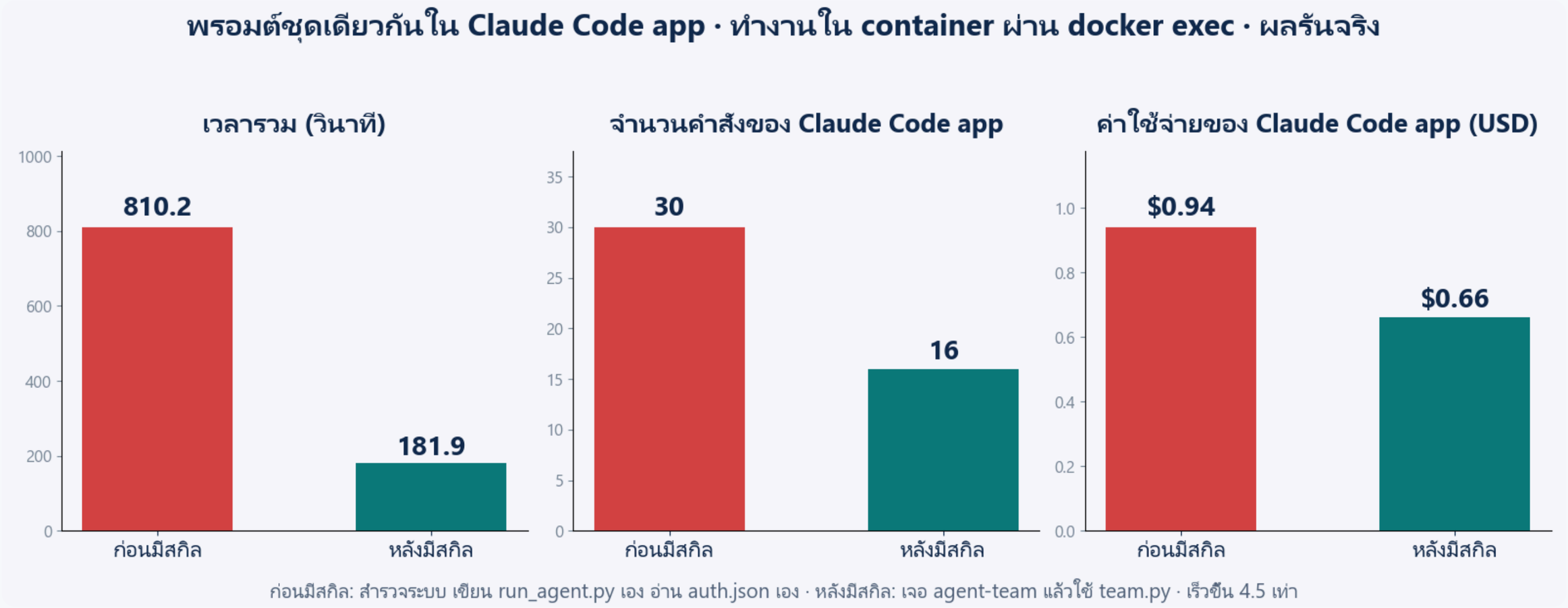
ตัวเลข Claude อ่านสดเวลา 07:50 ตัวเลข Codex มาจาก session ก่อนงานนี้ (งาน Codex ของทีมรันแบบ ephemeral ซึ่งไม่อัปเดตข้อมูลโควตา) ถ้าต้องการค่าสดใช้ `team.py quota --refresh`

ไฟล์อยู่ใน `/workspace/team-demo` : `a_claude.py` , `a_codex.py` , `final.py` , `test_final.py`

> ตรวจโฟลเดอร์สกิลแล้ว**เจอ agent-team** · อ่าน SKILL.md + graph.md · team.py check · เขียน graph แล้ว team.py run · โควตาจาก team.py quota

คำตอบจริงเป็นภาษาอังกฤษ แปลเป็นไทย

ขั้นที่ 3 · ก่อน vs หลังมีสกิล



ขั้นที่ 3 · ก่อน vs หลังมีสกิล

เร็วขึ้น 4.5 เท่า

	ก่อนมีสกิล	หลังมีสกิล
เวลารวม	810.2 วินาที	**181.9 วินาที**
คำสั่งของ Claude Code app	30 (ล้ม 9)	16 (ล้ม 4)
ค่าใช้จ่าย	$0.94	$0.66
วิธีเรียกแต่ละบัญชี	เขียน run_agent.py เอง	team.py run ของสกิล
ไฟล์ credential	ตรวจ .credentials.json · เปิด auth.json	ไม่แตะ
โทเค็นและโควตา	สคริปต์ที่เขียนเอง	ledger + team.py quota

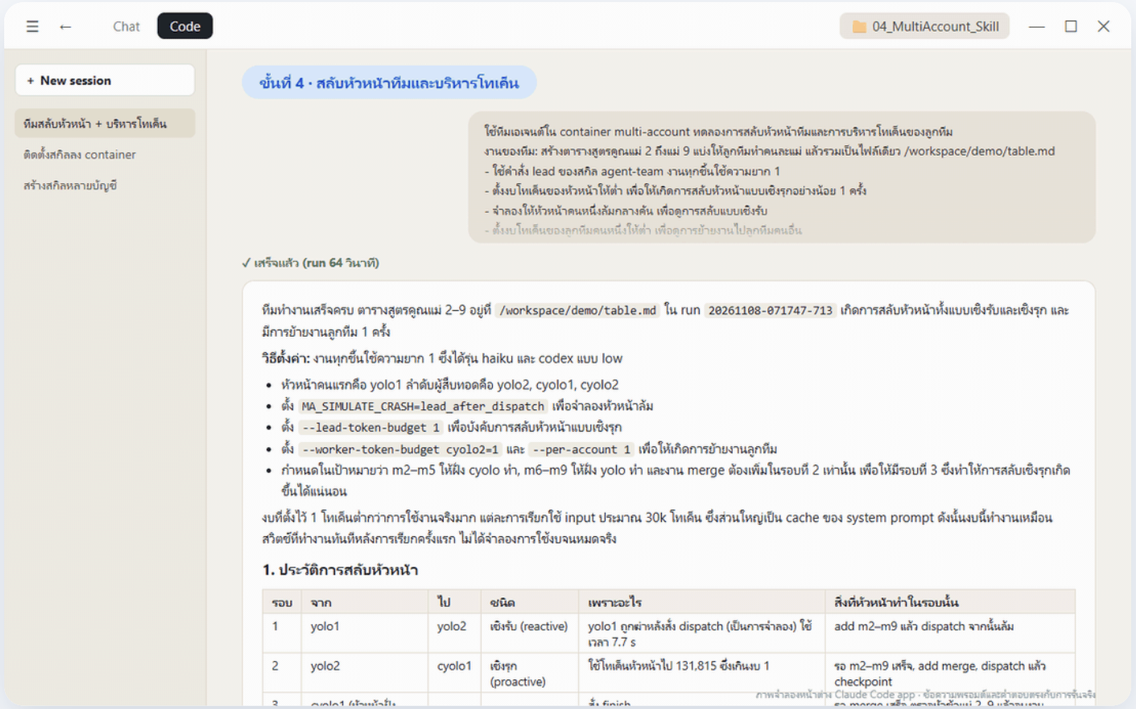
4

สลับหัวหน้าทีมและบริหารโทเค็น

team.py lead + ตัวจำลอง: หัวหน้าล้ม · หัวหน้าเกินงบ · ลูกทีมเกินงบ (~3 นาที)

ขั้นที่ 4 · 💻 Claude Code app

```
ใช้ทีมเอเจนต์ใน container multi-account ทดลองการสลับหัวหน้าทีมและการบริหารโทเค็นของลูกทีม
งานของทีม: สร้างตารางสูตรคูณแม่ 2 ถึงแม่ 9 แบ่งให้ลูกทีมทำคนละแม่ แล้วรวมเป็นไฟล์เดียว
/workspace/demo/table.md
- ใช้คำสั่ง lead ของสกิล agent-team งานทุกชิ้นใช้ความยาก 1
- ตั้งงบโทเค็นของหัวหน้าให้ต่ำ เพื่อให้เกิดการสลับหัวหน้าแบบเชิงรุกอย่างน้อย 1 ครั้ง
- จำลองให้หัวหน้าคนหนึ่งล้มกลางคัน เพื่อดูการสลับแบบเชิงรับ
- ตั้งงบโทเค็นของลูกทีมคนหนึ่งให้ต่ำ เพื่อดูการย้ายงานไปลูกทีมคนอื่น
สรุปเป็นตาราง: ประวัติการสลับหัวหน้า (จากใคร ไปใคร เพราะอะไร) การย้ายงานของลูกทีม ผลงานครบหรือไม่ และ
สถิติแยกตามบัญชี
```



การทำจำลองหน้าส่วน Claude Code app : ข้อความพร้อมตัวอย่างตรงกับการขึ้นจริง

ขั้นที่ 4 · ผลจริง · run 64 วินาที · 12 การเรียก

เหตุการณ์	เกิดอะไรขึ้น
สลับ**เชิงรับ**	yolo1 ล้ม (จำลอง) → yolo2 · งาน 8 ชิ้นรันต่อจนเสร็จ ไม่ถูกเพิ่มซ้ำ
สลับ**เชิงรุก**	yolo2 เกินงบหัวหน้า → cyolo1 (หัวหน้าฝั่ง Codex) จบงาน
ย้ายงานลูกทีม	cyolo2 เกินงบ → หยุดรับงาน · m5 ย้ายไป cyolo1

ตารางสูตรคูณครบ 8 แม่ 96 แถว ถูกทุกแถว · งบ 1 โทเค็นคือสวิตช์จำลอง

✻ Claude · ขั้นที่ 4 · สลับหัวหน้าทีมและบริหารโทเคิน · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ทีมทำงานเสร็จครบ ตารางสูตรคูณแม่ 2–9 อยู่ที่ `/workspace/demo/table.md` ใน run `20261108-071747-713` เกิดการสลับหัวหน้าทั้งแบบเชิงรับและเชิงรุก และมีการย้ายงานลูกทีม 1 ครั้ง

วิธีตั้งค่า: งานทุกชิ้นใช้ความยาก 1 ซึ่งได้รุ่น haiku และ codex แบบ low

- หัวหน้าคนแรกคือ yolo1 ลำดับผู้สืบทอดคือ yolo2, cyolo1, cyolo2
- ตั้ง `MA_SIMULATE_CRASH=lead_after_dispatch` เพื่อจำลองหัวหน้าล้ม
- ตั้ง `--lead-token-budget 1` เพื่อบังคับการสลับหัวหน้าแบบเชิงรุก
- ตั้ง `--worker-token-budget cyolo2=1` และ `--per-account 1` เพื่อให้เกิดการย้ายงานลูกทีม
- กำหนดในเป้าหมายว่า m2–m5 ให้ฝั่ง cyolo ทำ, m6–m9 ให้ฝั่ง yolo ทำ และงาน merge ต้องเพิ่มในรอบที่ 2 เท่านั้น เพื่อให้มีรอบที่ 3 ซึ่งทำให้การสลับเชิงรุกเกิดขึ้นได้แน่นอน

งบที่ตั้งไว้ 1 โทเคินต่ำกว่าการใช้งานจริงมาก แต่ละการเรียกใช้ input ประมาณ 30k โทเคิน ซึ่งส่วนใหญ่เป็น cache ของ system prompt ดังนั้นงบนี้ทำงานเหมือนสวิตช์ที่ทำงานทันทีหลังการเรียกครั้งแรก ไม่ได้จำลองการใช้งบจนหมดจริง

1. ประวัติการสลับหัวหน้า

รอบ	จาก	ไป	ชนิด	เพราะอะไร	สิ่งที่หัวหน้าทำในรอบนั้น
1	yolo1	yolo2	เชิงรับ (reactive)	yolo1 ถูกฆ่าหลังสั่ง dispatch (เป็นการจำลอง) ใช้เวลา 7.7 s	add m2–m9 แล้ว dispatch จากนั้นล้ม
2	yolo2	cyolo1	เชิงรุก (proactive)	ใช้โทเคินหัวหน้าไป 131,815 ซึ่งเกินงบ 1	รอ m2–m9 เสร็จ, add merge, dispatch แล้ว checkpoint
3	cyolo1 (หัวหน้าฝั่ง Codex)	—	—	สั่ง finish	รอ merge เสร็จ ตรวจหัวข้อแม่ 2–9 แล้วจบงาน

งานทั้ง 8 ชิ้นที่ yolo1 สั่งไว้ก่อนล้มยังรันต่อจนเสร็จ และไม่มีงานไหนถูกเพิ่มซ้ำ

2. การย้ายงานของลูกทีม

เวลา	เหตุการณ์	รายละเอียด
07:18:06	`worker_pause`	cyolo2 หยุดรับงานใหม่ หลังทำ m3 เสร็จ (ใช้ไป 32,691 โทเคิน เกินงบ 1)
07:18:06	`worker_switch`	งาน m5 ถูกย้ายจาก cyolo2 ไปให้ cyolo1 ทำ

3. ผลงานครบหรือไม่: ครบ

งาน	m2	m3	m4	m5	m6	m7	m8	m9	merge
ผู้ทำ	cyolo1	cyolo2	cyolo1	cyolo1 (ย้ายมา)	yolo2	yolo1	yolo2	yolo1	yolo2
สถานะ	done	done	done	done	done	done	done	done	done

ไฟล์มี 138 บรรทัด หัวข้อ `## แม่ 2` ถึง `## แม่ 9` ครบ มีแถวสูตรคูณ 96 แถว (8 แม่ × 12) ผมตรวจผลคูณทุกแถวด้วยสคริปต์แล้ว ไม่พบแถวที่ผิด

4. สถิติแยกตามบัญชี (12 การเรียก)

บัญชี	บทบาทใน run	ครั้ง	สำเร็จ / ล้ม	input (cache read)	output	reasoning	$ list	เวลารวม s
yolo1	หัวหน้ารอบ 1 + ทำ m7, m9	3	2 / 1 (ล้มคือรอบที่ถูกฆ่า)	118,254 (83,833)	1,162	419	0.0083	20.4
yolo2	หัวหน้ารอบ 2 + ทำ m6, m8, merge	4	4 / 0	334,400 (261,075)	4,013	1,287	0.0193	60.5
cyolo1	หัวหน้ารอบ 3 + ทำ m2, m4, m5	4	4 / 0	187,956 (162,560)	1,244	0	—	50.5
cyolo2	ทำ m3 แล้วหยุดรับงาน	1	1 / 0	32,442 (28,288)	249	0	—	11.2
รวม		**12**	**11 / 1**	**673,052 (535,756)**	**6,668**	**1,706**	**0.0276**	**142.5**

ค่า `$ list` คือราคาตามรายการของฝั่ง Claude ไม่ใช่ยอดที่จ่ายจริง ฝั่ง Codex ไม่มีข้อมูลค่าใช้จ่าย หลังจบงาน โควตาทุกบัญชียังเหลือมาก (ใช้ไปไม่เกิน 22%/สัปดาห์)

ไฟล์ทั้งหมดของ run (`handoff.md`, `board.md`, `state.json`, `ledger.jsonl`) อยู่ใน `/workspace/.agent-team/20261108-071747-713/`

5

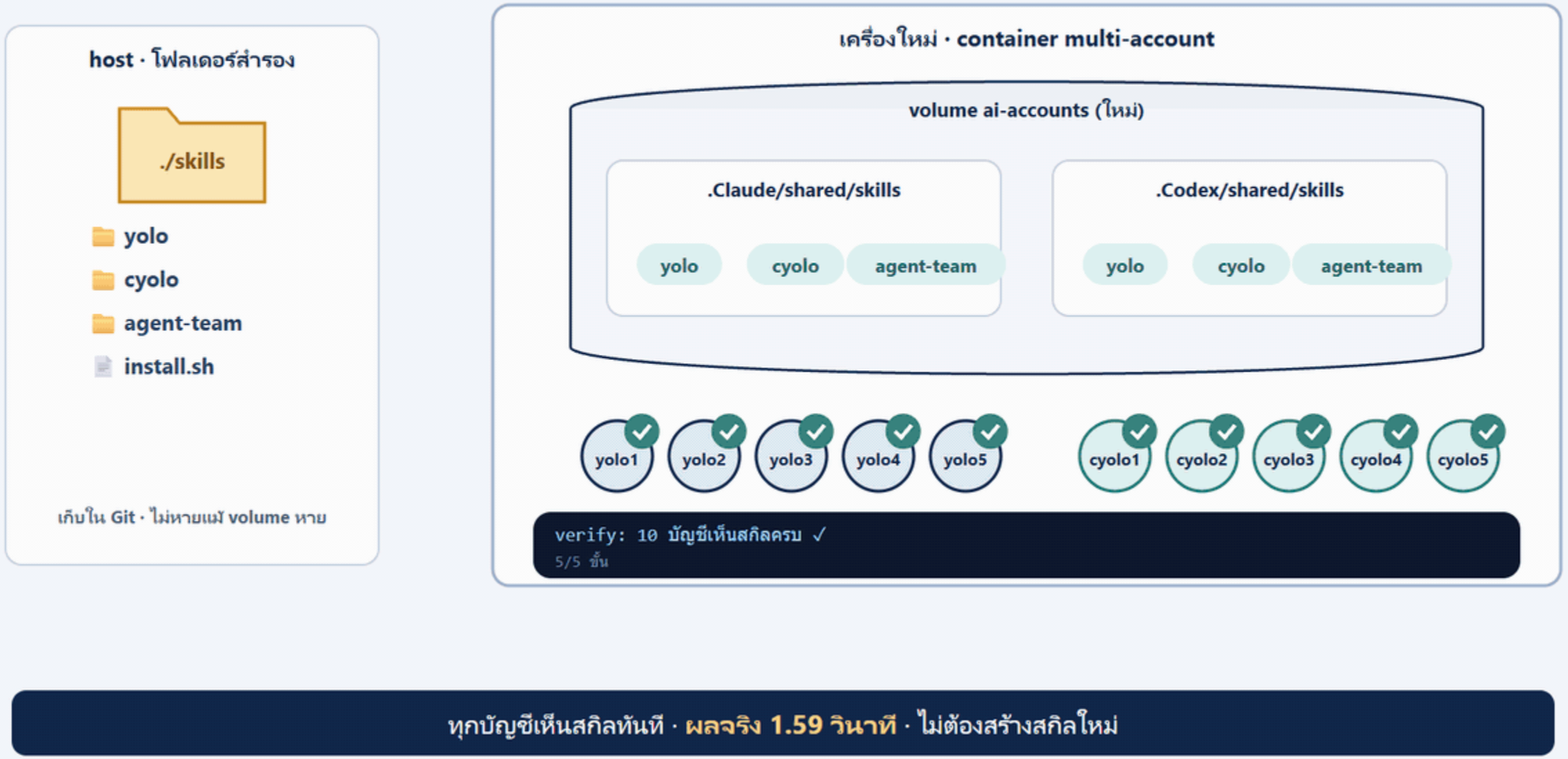
ติดตั้งบนเครื่องที่ยังไม่มี volume

volume ใหม่ไม่มีสกิล → copy โฟลเดอร์สำรอง → **1.59 วินาที**

ขั้นที่ 5 · แนวคิด

ติดตั้งสกิล บนเครื่องที่ยังไม่มี volume

ต้นฉบับสกิลอยู่นอก container จึงนำไปวางใหม่ได้ทุกเมื่อ



ทุกบัญชีเห็นสกิลทันที · ผลจริง 1.59 วินาที · ไม่ต้องสร้างสกิลใหม่

ขั้นที่ 5 · 💻 Claude Code app

จำลองเครื่องใหม่ที่ยังไม่มี volume ai-accounts แล้วติดตั้งสกิลจากโฟลเดอร์สำรอง ./skills
- สร้าง volume ai-accounts-new และ container multi-account-new จาก image multi-account:1.0 (พอร์ต 8889 · --privileged)
- ตรวจว่าใน container ใหม่ยังไม่มีสกิล yolo, cyolo, agent-team
- ติดตั้งจาก ./skills (docker cp + install.sh) และจับเวลาตั้งแต่เริ่มคัดลอกจนตรวจเสร็จ
- ตรวจว่าทุกบัญชี yolo1–5 และ cyolo1–5 มองเห็นสกิลครบ และ team.py check ทำงานได้ (บัญชีใหม่ยังไม่ได้ล็อกอินเป็นเรื่องปกติ)
- เสร็จแล้วลบ container multi-account-new และ volume ai-accounts-new ทิ้ง ห้ามแตะ container multi-account และ volume ai-accounts
สรุปเป็นตาราง ก่อนติดตั้ง | หลังติดตั้ง และเวลาที่ใช้ แล้วหยุด



ขั้นที่ 5 · ผลจริง

✻ **Claude** · ขั้นที่ 5 · ติดตั้งบนเครื่องที่ยังไม่มี volume · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ติดตั้งสกิลจาก `./skills` บนเครื่องใหม่สำเร็จ ทั้ง 10 บัญชีมองเห็นสกิลครบทั้ง 3 ตัว และ `team.py check` ทำงานได้ แต่ install.sh รอบแรกล้มกับบัญชี yolo1–5 ผมจึงแก้ `skills/install.sh` แล้วทดสอบใหม่ตั้งแต่ต้นบน volume ว่าง

รายการ	ก่อนติดตั้ง	หลังติดตั้ง
สกิลใน `.Claude/shared/skills`	มีแค่ `shared-check`	`shared-check` , `yolo` , `cyolo` , `agent-team`
สกิลใน `.Codex/shared/skills`	มีแค่ `shared-check`	`shared-check` , `yolo` , `cyolo` , `agent-team`
บัญชี yolo1–5 ที่เห็นสกิลทั้ง 3	0/5	5/5 ✅
บัญชี cyolo1–5 ที่เห็นสกิลทั้ง 3	0/5	5/5 ✅
`SKILL.md` ที่อ่านได้ (10 บัญชี × 3 สกิล)	0/30	30/30 ✅
`team.py check`	ยังไม่มีไฟล์	ทำงานได้ · บัญชีใหม่ขึ้น "ไม่ได้ล็อกอิน" เป็นเรื่องปกติ

เวลาตั้งแต่เริ่มคัดลอกจนตรวจเสร็จ: 1.59 วินาที (`docker cp` 0.39 · `install.sh` 0.55 · ตรวจทุกบัญชี 0.65)

แก้ install.sh: ใน volume ใหม่ `yoloN/skills` เป็นโฟลเดอร์จริง ไม่ใช่ symlink เหมือน volume เดิม จึงต้องสร้าง link รายสกิลให้ทั้ง yolo และ cyolo แบบเดียวกับ wrapper ของ image

เก็บกวาด: ลบ container `multi-account-new` และ volume `ai-accounts-new` แล้ว ไม่ได้แตะ container `multi-account` และ volume `ai-accounts`

1.59 วินาที · 30/30 SKILL.md อ่านได้

เจอบั๊กจริง: volume ใหม่ yoloN/skills เป็นโฟลเดอร์จริง install.sh รอบแรกจึงลิงก์ให้แค่ Codex · Claude แก้แล้วทดสอบใหม่

ทดสอบบนเครื่องใหม่จริงจำเป็น เพราะเครื่องเดิมอาจซ่อนปัญหา

สรุป

ไม่มีสกิล vs มีสกิล

	ไม่มีสกิล	มีสกิล
วิธีเรียกแต่ละบัญชี	ลองผิดลองถูก เปิด --help	สคริปต์ใส่ flag ถูกและตรวจผลให้
บัญชีไหนใช้ได้	auth status (ไม่ตรงจริง)	check ด้วยการเรียกจริง
การทำงานร่วมกัน	สั่งทีละตัว ยัดข้อมูลลง prompt	relay · fan-out · graph · review · board · ขั้นสูง
สถิติ	ต้องเดา	ledger + โควตาพร้อมเวลาของข้อมูล
โควตาหมด	งานค้างหรือหาย	สลับหัวหน้าและย้ายงาน งานไม่หาย
เครื่องใหม่	สร้างใหม่	copy โฟลเดอร์สำรอง 1.59 วินาที
งานทีมชุดเดียวกัน	810.2 วินาที	181.9 วินาที

ข้อควรรู้

ข้อควรรู้

ล็อกอินเองเสมอ

สกิลไม่ล็อกอินแทน และไม่อ่าน .credentials.json / .claude.json / auth.json

ข้าม permission

ใช้กับงานที่อนุญาตเท่านั้น ตรวจผลของ agent อื่นซ้ำเสมอ

ไม่ retry อัตโนมัติ

งานที่ล้มหลังใช้โทเค็นแล้วหยุดและรายงาน

Git Bash แปลง path

docker exec จาก Git Bash ใช้ `//root/...` หรือ PowerShell

แก้ที่ต้นฉบับ

แก้ใน skills/ แล้วรัน install.sh ใหม่ (สำรองของเดิมให้)

macore.py 2 สำเนา

ใน yolo และ cyolo แก้แล้วต้องให้ตรงกัน (install.sh ตรวจให้)

คำสั่งที่ใช้บ่อย

ใน container

```
T=/root/ai-accounts/.Claude/shared/skills/agent-
team/scripts/team.py
python3 $T check            # บัญชีไหนใช้ได้ (เรียกจริง)
python3 $T quota            # โควตาทุกบัญชี + เวลาของข้อมูล
python3 $T run graph.json   # รันกราฟงานจนจบ
python3 $T status           # ภาพรวมทีม
python3 $T stats            # สถิติตามบัญชี / รุ่น / run
```

ติดตั้งเอง (จากโฟลเดอร์บท)

```
docker cp ./_solution/skills multi-account:/tmp/skills
docker exec multi-account bash /tmp/skills/install.sh
```

จาก host

```
docker exec multi-account cat /root/ai-
accounts/.Claude/shared/skills/agent-team/SKILL.md
docker exec multi-account python3 /root/ai-
accounts/.Claude/shared/skills/agent-team/scripts/team.py
stats
```

เช็กลิสต์

ทำครบหรือยัง?

- [ ] มีโฟลเดอร์ `skills/` ที่มี yolo, cyolo, agent-team (ขั้นที่ 1)
- [ ] ทุกบัญชีเห็นสกิลครบ (ขั้นที่ 2)
- [ ] งานทีมชุดเดิมเร็วขึ้น และตัวเลขมาจาก ledger (ขั้นที่ 3)
- [ ] เห็นการสลับหัวหน้าทั้งเชิงรุก/เชิงรับ และการย้ายงานลูกทีม (ขั้นที่ 4)
- [ ] ติดตั้งบน volume ใหม่จากโฟลเดอร์สำรองได้ (ขั้นที่ 5)

สรุป

สอนครั้งเดียว ทุกบัญชีทำงานเป็นทีม

ความรู้อยู่ใน **สกิล** · สถานะอยู่ใน **ไฟล์** · ต้นฉบับอยู่ใน **โฟลเดอร์สำรอง**
หัวหน้าหรือลูกทีมโควตาหมด งานก็ไม่หาย

รายละเอียดทุกขั้นตอนและพรอมต์เต็มอยู่ใน README.md

